

Extra Appendix: Measurement Details and Earlier Experiments

Soheila Jafari Khouzani

Fall 2026

This supplement preserves supporting measurement explanations and plots omitted from the main proposal. It contains the consumer-offset example, measurement coverage, twelve five-minute-evaluation trials, detailed plots for the eight ten-minute-evaluation trials, and the plots from the historical 24-trial scaling campaign. These are separate experiment groups; additional plots do not represent additional trials. The five-minute-evaluation trials used one minute of warm-up and two minutes of drain, for eight minutes in total. The historical campaign used different durations and an earlier deployment, as stated in its section. The main proposal retains the balanced calibration and compact ten-minute result figures.

Completion latency ends when application processing finishes, before offset-commit acknowledgment. P99 uses distinct evaluation messages completed by the drain cutoff. Unfinished percentages and SLO violations use all successfully published evaluation messages; the one-second SLO also counts overdue unfinished messages. Throughput counts distinct completions during evaluation, including any queued warm-up messages. Consumer resource totals cover evaluation and drain. Missing monitoring intervals remain gaps.

1 Consumer Position and Offset Commits: Example

This illustrative example concerns one partition processed sequentially. An offset identifies a record's position in that partition. The consumer application has received records 100–104. Records 100 and 101 have finished processing, record 102 is being processed, and records 103 and 104 are waiting. Kafka has most recently acknowledged a committed offset of 100, meaning that progress through record 99 was saved.

Progress measure	Value	Meaning at this instant
Consumer position	105	Records through 104 were received by the consumer application; 105 is next to be received.
Completion frontier	102	Records through 101 have finished processing; 102 is the next unfinished record.
Committed offset	100	Kafka's saved restart point is 100. Finished work after record 99 has not yet been committed.

The application has therefore received five records but finished only two of them. If it crashes now, before any newer commit succeeds, normal consumption resumes from offset 100. Records 100 and 101 may be processed again even though their application processing had finished. A later commit can advance the restart point only through successfully completed work. Reading records and committing offsets do not delete those records from Kafka; retention settings govern their removal.

In Kafka client terminology, records are *returned* when the client's receive call hands them to the consumer application. This means received by the application, not sent back to Kafka and not necessarily processed.

2 Measurement Coverage

Table 1: Metric coverage and limitations.

Metric family	Evidence in results	Interpretation
Partition lag and total lag	Main lag plots, diagnostic figure and lag-summary tables; older five-minute counterparts in the separate extra appendix.	Offsets, not guaranteed record counts.
Processing backlog and growth	Main backlog-growth plots and acknowledged-unfinished plots; growth is also retained for consumer-position lag.	Completion frontier and returned-record position have different definitions.
Maximum and mean partition lag	Bottom row of each diagnostic figure and tables of sampled maxima.	Maximum and mean are taken across partitions at each valid observation.
Lag skew and window-mean skew	Runtime-state row shows window-mean skew; the summary table and separate diagnostic plot retain raw skew.	High relative skew can coexist with small absolute lag.
Hot set, persistence score and persistent-hot set	Persistent-hot row, detector settings and interpretation of early versus late observations.	Exploratory diagnostic, not a validated controller trigger.
Window-mean backlog and runtime state	Runtime-state figure and timestamped CSV/JSON, including persistent-hot partition IDs.	Reconstructed from retained observations; no adaptive controller was evaluated.
Completion mean and p99	Main outcome tables for each workload.	Calculated from completed evaluation messages through drain; p99 is not an average of rolling percentiles.
Unfinished work and completion deadlines	Unfinished percentage and one-second SLO violations in main tables; 0.5-second sensitivity in appendix.	Overdue unfinished messages count as misses; they have no fabricated completion latency.
Input rate and throughput	Main throughput plots and outcome tables; acknowledged input rate in the text.	Throughput counts each distinct completed message once.
Actual CPU and resident memory	Main process traces and observed resource integrals.	Consumer processes only; no full-cluster energy or monetary-cost claim.
Requested CPU and memory cost	Main resource tables, integrated over evaluation plus drain; detailed resource-cost figures retained with records.	Partial request coverage is unavailable for full-run comparison, not zero.
Interruption, transition backlog and recovery	Main intervention-cost tables, with operational endpoints identified.	Recovery is first threshold confirmation while input continues; not permanent stability.
Correctness and measurement quality	Identity reconciliation, handover validation, clock limitations and explicit coverage discussion.	Checks cover the completed stateless synthetic trials.
Controller overhead and hot-key splitting	Explicitly identified as unevaluated future measurements.	Neither was evaluated in these trials; stateful transfer is outside the primary scope.

Contents

1	Consumer Position and Offset Commits: Example	2
2	Measurement Coverage	3
3	Five-Minute Evaluation: Ownership and Intervention Results	5
3.1	Relation to the ten-minute comparison	5
3.2	Partition ownership at the reference rate	5
3.3	Live mitigation under concentrated partition ownership	10
3.3.1	Adding consumers with explicit redistribution	10
3.3.2	Redistribution without additional consumers	10
3.4	Resource requirements, interruption and recovery	14
3.4.1	Processing interruption and continuing-input recovery	15
4	Supporting Ownership Diagnostics	17
5	Supporting Measurements for Live Interventions	21
5.1	Monitoring coverage and interpretation	21
5.2	Persistent-hot and partition-lag diagnostics	36
6	Completion-Deadline Sensitivity	38
7	Detailed Plots: Ten-Minute Evaluation	39
8	Historical Scaling Campaign: Preserved Plots	52

3 Five-Minute Evaluation: Ownership and Intervention Results

This extra appendix retains the four fixed-ownership trials and the two four-trial intervention blocks with five minutes of evaluation. Each trial also includes one minute of warm-up and two minutes of drain. These twelve trials are distinct from the twelve balanced-workload trials at different input rates and the twelve ten-minute intervention trials in the main proposal.

3.1 Relation to the ten-minute comparison

The main proposal’s skewed-workload results use ten minutes of evaluation, whereas the trials in this appendix use five. In the ten-minute comparison, redistribution within three consumers met the recovery criterion at evaluation seconds 372 and 396, beyond the shorter evaluation period. That comparison also includes normal Kafka scaling as a separate configuration. Trial dates, high-traffic partition identities and the monitoring implementation changed between the groups, so differences cannot be attributed to evaluation duration alone. The monitoring correction is described in Section 5.

3.2 Partition ownership at the reference rate

Four exploratory trials examined the effect of initial ownership at the same aggregate rate of 700 messages/s. The same 12 partition IDs, 0–11, received approximately 80% of input across 60 partitions. Every consumer owned 20 partitions. One layout assigned four high-traffic partitions to each consumer; the others assigned all 12 to Consumer 0, Consumer 1 or Consumer 2. These partitions are designated high-traffic from the input configuration; a processing hotspot is identified from the observed backlog and growth. Ownership was fixed before traffic, with no scaling or live reassignment.

Each trial lasted eight minutes: one minute of warm-up, five minutes of evaluation and two minutes of drain. Each trial included 210,000 messages produced during evaluation. Exact initial ownership was verified before traffic, and valid observations retained the expected owners and process identities. Recorded consumer pod identities and nodes matched across the four trials, without an additional rule forcing pods onto particular nodes. Table 2 reports all four outcomes in the figure display order. Actual execution order was distributed ownership, Consumer 0, Consumer 2 and Consumer 1; later layouts were selected after earlier observations.

High-traffic placement	Maximum total backlog (offsets)	Growth (offsets/s)	p99 (s)	Unfinished (%)	Throughput (msg/s)
Four per consumer	59	−0.020	0.193	0	700.08
All 12 on Consumer 0	141	+0.154	0.249	0	699.99
All 12 on Consumer 1	58	−0.013	0.248	0	699.99
All 12 on Consumer 2	102,475	+288.319	233.857	32.286	413.15

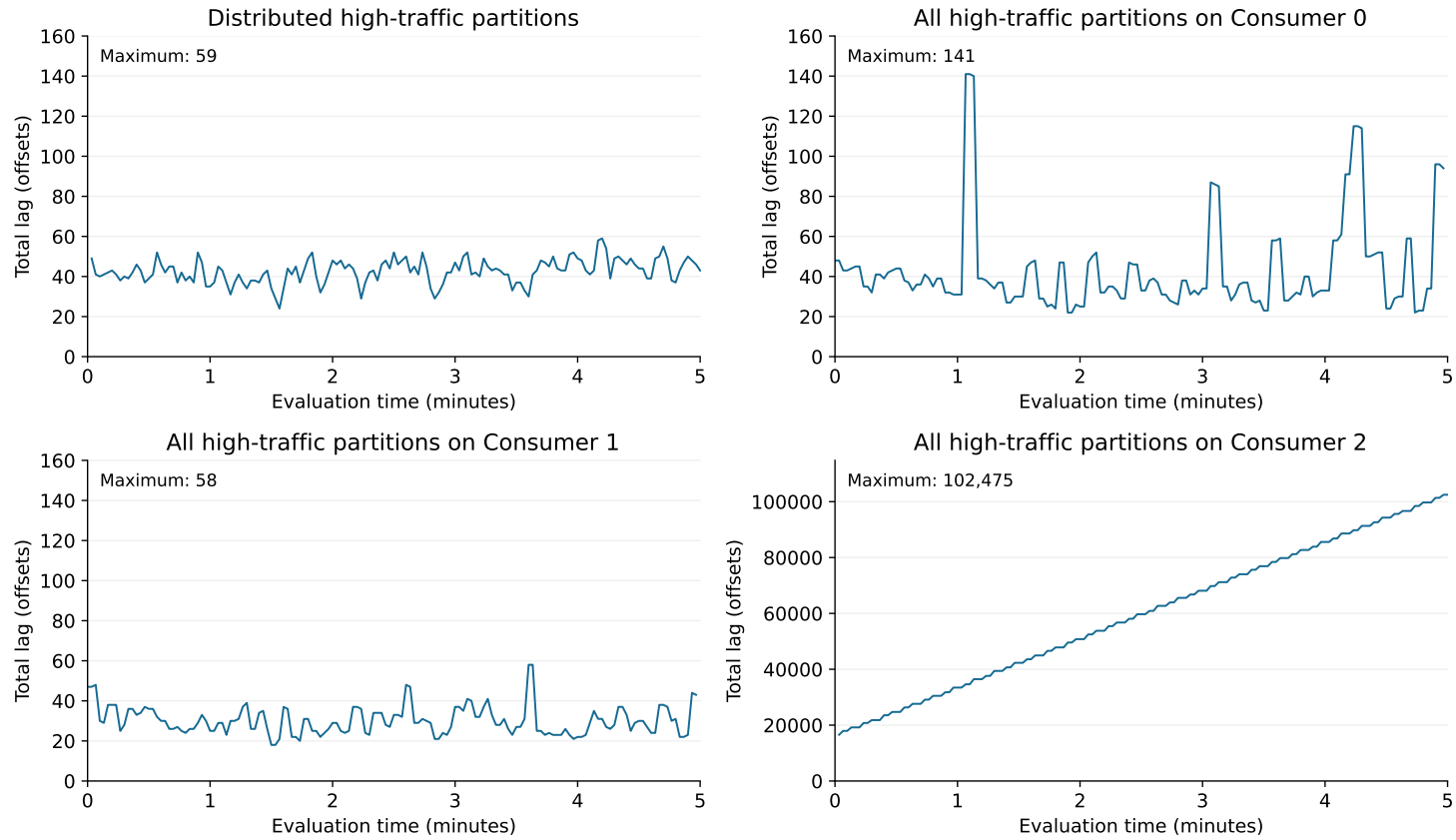
Table 2: Results under four fixed partition assignments. Growth summarizes valid processing-backlog measurements during evaluation; p99 and unfinished messages are evaluated through drain.

Concentration on Consumer 2 produced sustained backlog growth and left 67,801 of 210,000 evaluation messages unfinished after drain. Its completion p99 was 233.857 seconds and evaluation throughput was 413.15 messages/s. The other three layouts completed all evaluation messages by the cutoff and sustained approximately 700 completions/s during evaluation. Thus, a reference rate with low balanced-workload lag can still coincide with a bottleneck under a concentrated assignment. Concentration alone did not cause sustained accumulation in every observed layout.

Figures 1 and 2 show the backlog difference, while Figures 3 and 4 relate it to processing activity. In the Consumer 2 concentration trial, that consumer averaged 1.024 process CPU cores, compared with 0.152 and 0.118 cores for Consumers 0 and 1. These observations do not isolate hardware capacity, scheduling or contention as the cause. The Consumer 2 evaluation also began with backlog accumulated during warm-up; that work remained in the system even though warm-up messages were excluded from latency of evaluation messages and unfinished-message outcomes.

Section 4 provides the supporting memory, lag-skew and lag-by-consumer comparisons.

Same 80/20 input, different ownership — 700 messages/s

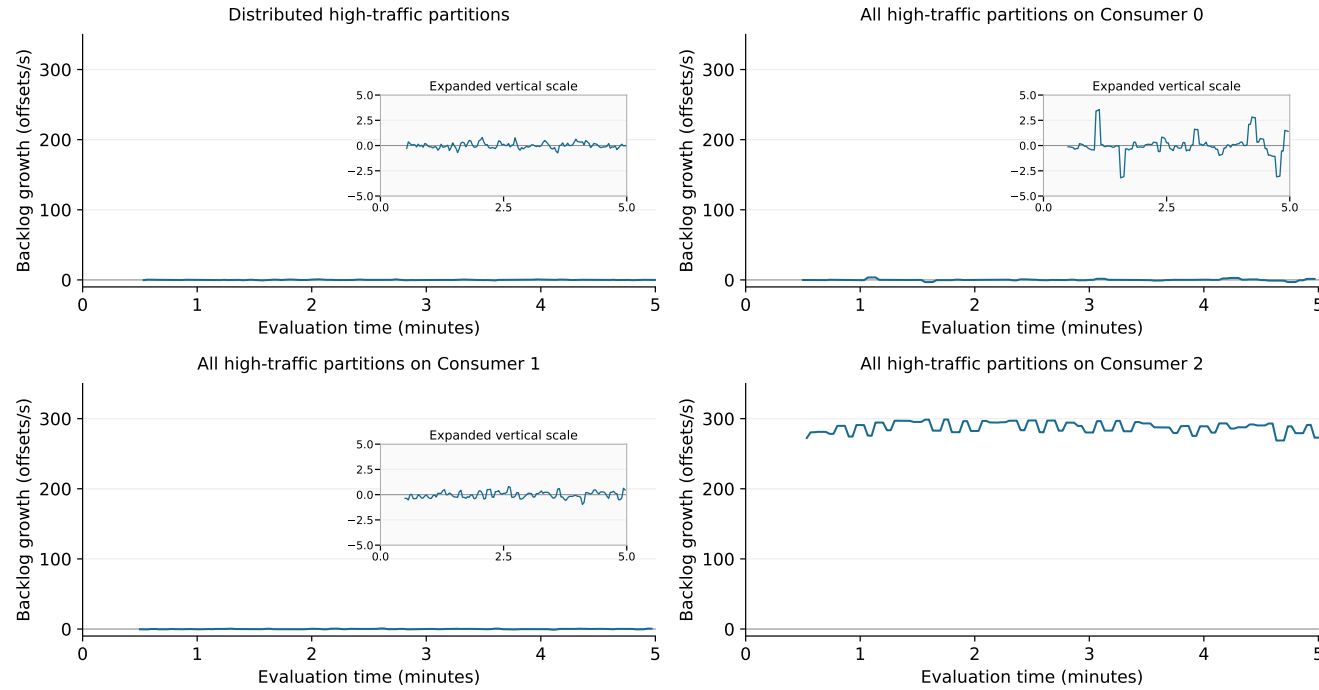


1 min warm-up + 5 min evaluation + 2 min drain. Panels use different vertical scales.

Figure 1: Total consumer-position lag during the five-minute evaluation of each fixed-ownership trial. The Consumer 2 concentration panel uses the explicitly labelled larger vertical scale. Panels show different initial layouts, not stages of a live reassignment. The full schedule also includes one minute of warm-up and two minutes of drain.

Processing-backlog growth

700 messages/s total · Three consumers · 80% of input across 12 of 60 partitions



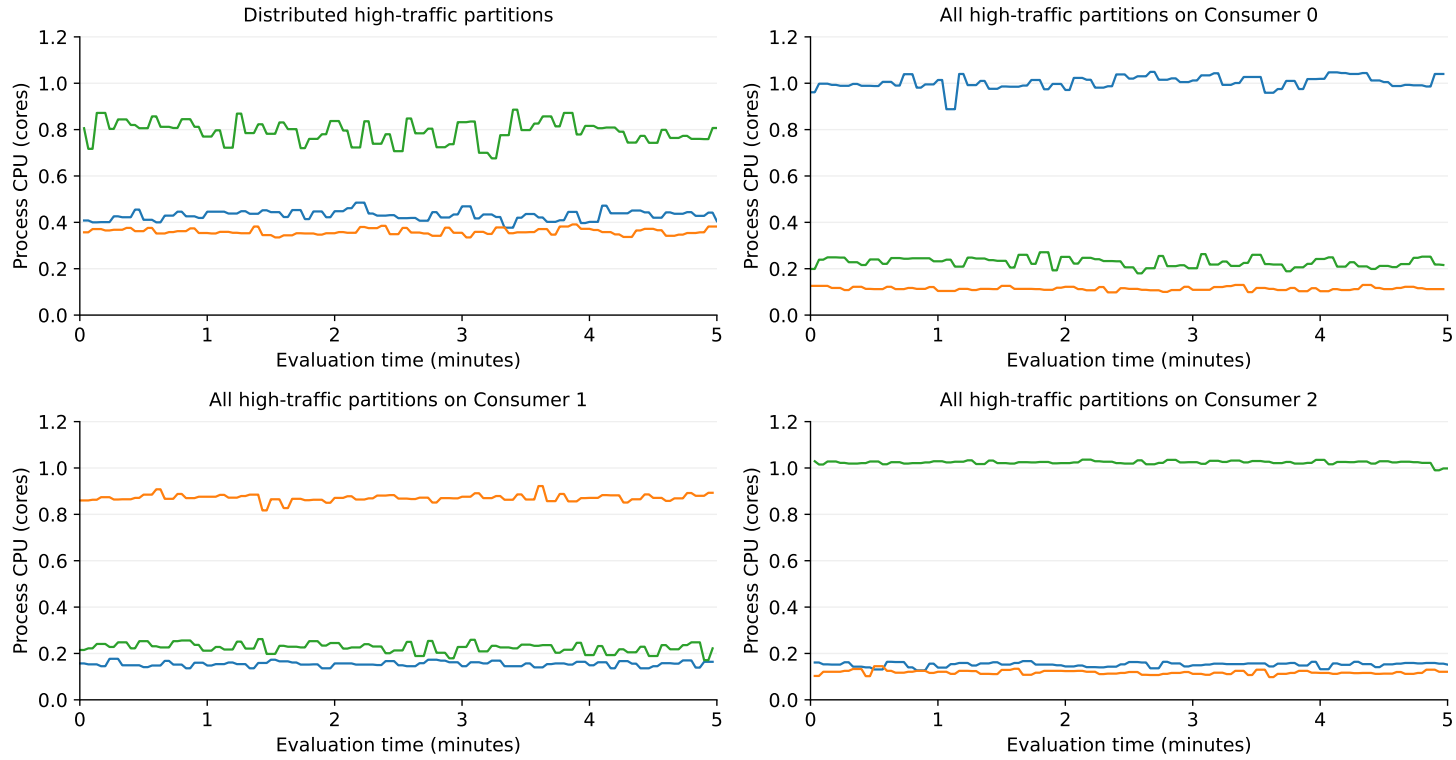
Five-minute evaluation. Main panels share the same vertical scale. Insets show the same data with a shared expanded scale. Growth uses a rolling 30-second window; gaps remain unavailable.

Figure 2: Rolling ten-second processing-backlog growth during evaluation. The display window was shortened from thirty seconds after the trials and applied consistently to all four layouts; complete valid windows are required after missing observations or ownership/offset resets. Main panels share a scale; labelled insets show small fluctuations. A positive growth rate indicates continued accumulation, even when the curve is approximately flat. Processing backlog is measured from the position up to which all preceding messages have completed. Consumer-position lag instead uses the position up to which the consumer has received records. The whole-evaluation summaries in Table 2 use covered intervals without bridging monitoring gaps.

Consumer CPU usage

700 messages/s total · Three consumers · 80% of input across 12 of 60 partitions

— Consumer 0 — Consumer 1 — Consumer 2



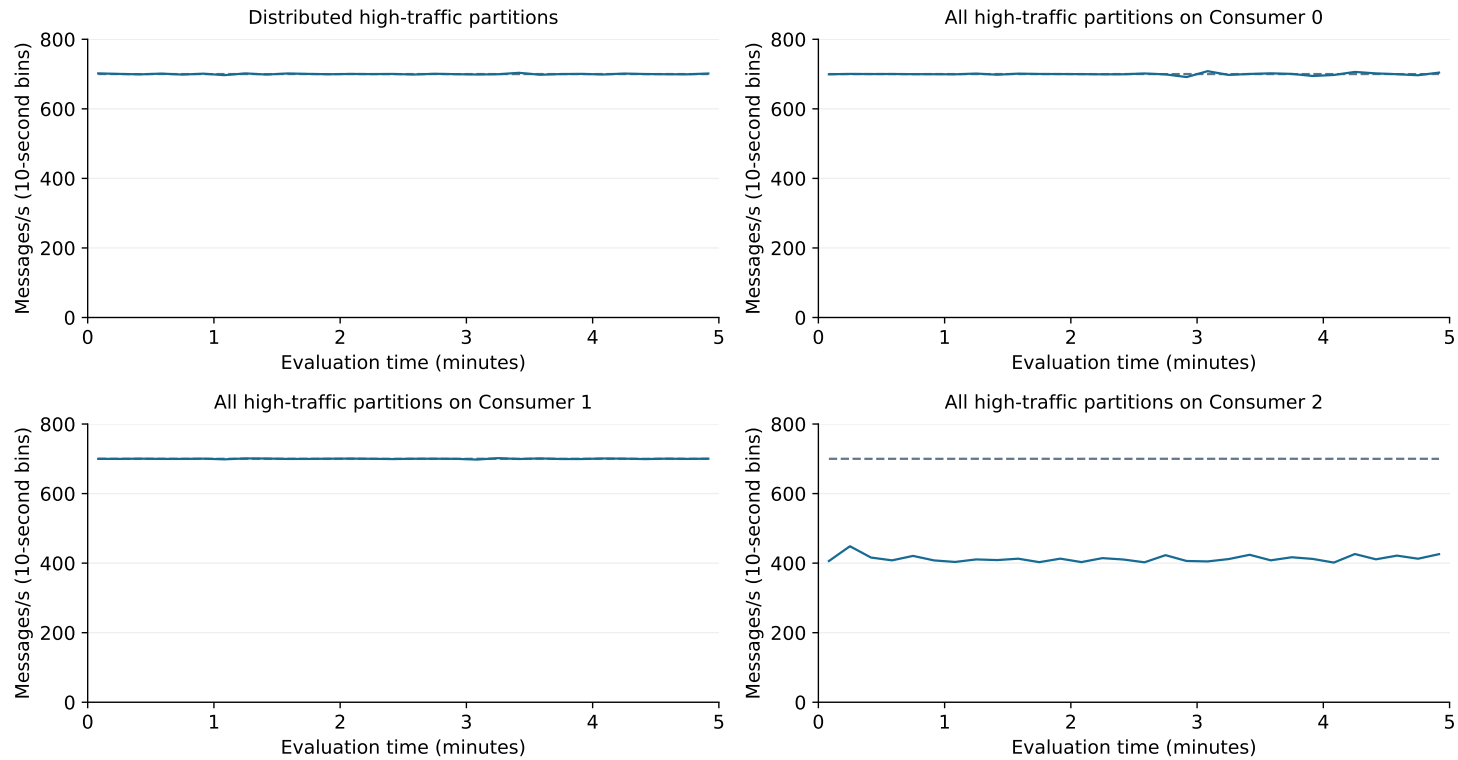
Five-minute evaluation. Main panels share the same vertical scale.
Process measurements; 1 CPU core corresponds to 100% CPU use.

Figure 3: Measured consumer-process CPU use during the five-minute evaluation, in cores. Panels share a scale and consistent consumer colours. These traces are process utilization, not requested CPU resources, whole-container or node utilization, or monetary charges. Shared-machine variation remains a possible influence.

Input and completion throughput

700 messages/s total · Three consumers · 80% of input across 12 of 60 partitions

-- Acknowledged input — Unique completions



Five-minute evaluation. Main panels share the same vertical scale.
Unique completions can include warm-up records finishing during evaluation.

Figure 4: Acknowledged input and distinct processing completions during evaluation. Completion throughput includes warm-up messages completed during evaluation; the latency and unfinished outcomes instead follow the messages produced during evaluation through drain. The Consumer 2 concentration trial completes fewer messages per second than arrive over the observed evaluation.

3.3 Live mitigation under concentrated partition ownership

The next eight trials tested whether an observed ownership bottleneck could be relieved during processing. We retained the reference input rate of 700 messages/s, three producers, 60 partitions, and the synthetic task of 2,000 SHA-256 iterations per message. Approximately 80% of traffic was directed to partitions 0–11. All twelve initially belonged to Consumer 2, reproducing the layout that accumulated backlog in the ownership calibration. The experiments therefore concern several busy partitions sharing one consumer, rather than a single hot key or a single indivisible overloaded partition.

Two separate four-trial blocks compared an unchanged assignment with a scheduled intervention. The first added three consumers and explicitly redistributed all 60 partitions, leaving two high-traffic and eight other partitions per consumer. The second redistributed ownership within the existing three consumers, leaving four high-traffic and sixteen other partitions per consumer. The first intervention changes both capacity and ownership; it is not a scaling-only experiment. Neither block uses an automatic decision policy or producer-side key splitting.

Each trial lasted eight minutes: one minute of warm-up, five minutes of evaluation, and two minutes of drain. In each trial, Kafka acknowledged successful publication of 210,000 messages produced during evaluation. The intervention was scheduled 60 seconds after evaluation began. Original consumer identities, nodes, resource requests and initial ownership were checked against the common reference; no additional rule forced pods onto specific nodes, and placement of added consumers remained controlled by Kubernetes. Each block used random-generation seeds 71 and 72 in the sequence unchanged Run 1, intervention Run 1, intervention Run 2, unchanged Run 2. This reverses the order of the unchanged and intervention runs across repetitions; it does not compare the execution order of different mitigation actions.

The transfers used the coordinated stop, transfer and restart procedure described in the proposal. Processing was stopped for handoff, the position up to which all preceding messages had completed and the partition owners were checked, and the new owners resumed from the recorded boundary. The observed identity, offset and handoff checks passed. These checks cover the observed handovers in the stateless synthetic trials. This experimental assignment mode is reported separately from the ordinary cooperative-sticky rebalancing evaluated in the proposal’s ten-minute comparison.

3.3.1 Adding consumers with explicit redistribution

Adding consumers and redistributing the busy partitions reduced recorded p99 latency in both comparisons (Table 3). P99 fell from 231.58 to 110.70 seconds in Run 1 and from 229.28 to 109.47 seconds in Run 2, reductions of approximately 52.2% in both. The unchanged trials left 66,695 and 65,178 evaluation messages unfinished, whereas both intervention trials completed all evaluation messages by the drain cutoff. Figure 5 presents these two outcomes together because the baseline p99 excludes its unfinished messages.

Figure 5 shows backlog falling after processing resumed under the new assignment. Distinct completion throughput during evaluation increased from approximately 416–419 to 756–757 messages/s. A completion rate above the 700-message/s input rate is possible while earlier queued messages are being cleared; it is not evidence that producers exceeded their configured rate. The combined intervention demonstrates a benefit relative to retaining this concentrated assignment, but it does not establish that six consumers were necessary. Both added capacity and changed ownership contributed to the intervention being tested.

3.3.2 Redistribution without additional consumers

Redistributing the same partitions among the three existing consumers also improved the recorded outcomes. P99 fell from 226.34 to 103.76 seconds in Run 1 and from 227.13 to 91.38 seconds in Run 2, reductions of 54.2% and 59.8%. The unchanged trials left 63,382 and 63,998 evaluation messages unfinished, while both redistribution trials completed all evaluation messages by the cutoff. Completion throughput rose from approximately 422 to 725–728 messages/s. Figure 6 shows the completion outcomes and available lag observations.

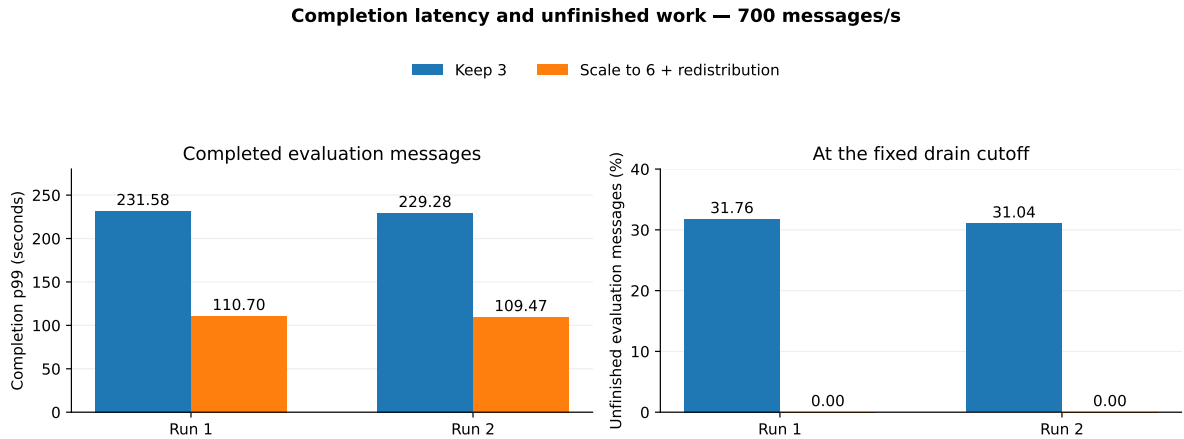
These findings support a specific interpretation: changing whole-partition ownership can use capacity that is already available elsewhere in the consumer group. Before intervention, one consumer handled all twelve high-traffic partitions; after intervention, their processing was shared among three consumers through separate partition ownership. No partition was divided among simultaneous consumers, and no

records were rerouted by key. Consequently, this result does not resolve the different case in which one partition remains overloaded even when it has a consumer to itself.

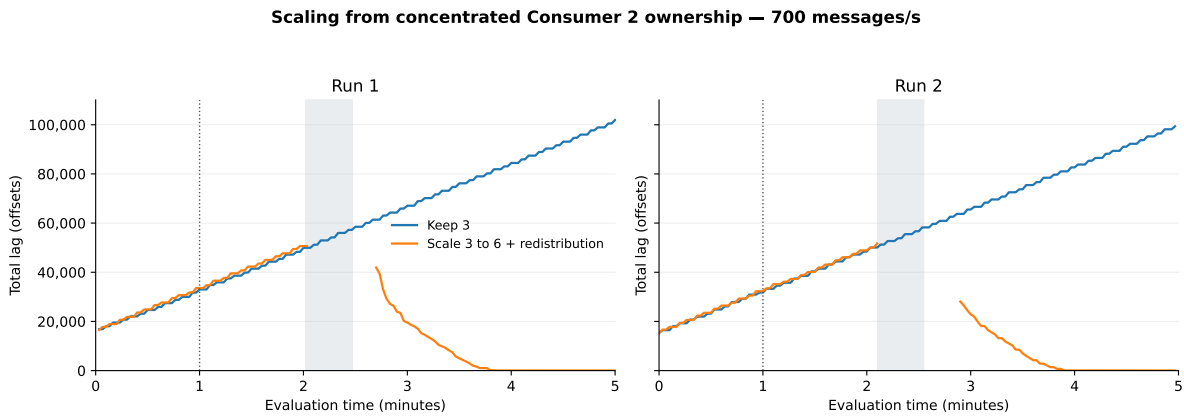
Table 3 reports the two separate four-trial comparisons; it does not directly rank the two interventions. P99 and unfinished percentages follow the 210,000 evaluation messages through drain. Throughput includes any warm-up messages completed during evaluation.

Response	Run	p99 (s)	Unfinished (%)	Misses > 1 s (%)	Throughput (msg/s)
Scaling with targeted redistribution					
Keep 3	1	231.58	31.760	83.42	415.93
Targeted scale 6	1	110.70	0.000	57.01	757.32
Keep 3	2	229.28	31.037	83.24	418.90
Targeted scale 6	2	109.47	0.000	58.17	755.63
Redistribution within three consumers					
Keep 3	1	226.34	30.182	83.41	422.08
Redistribute 3	1	103.76	0.000	61.59	725.25
Keep 3	2	227.13	30.475	83.24	421.76
Redistribute 3	2	91.38	0.000	60.84	727.54

Table 3: Five-minute completion outcomes and one-second SLO violations. The two blocks have separate baselines.



Each trial: 1 min warm-up + 5 min evaluation + 2 min drain; 210,000 evaluation messages. P99 is a whole-run completed-cohort percentile, not an average of rolling p99 values.



1 min warm-up + 5 min evaluation + 2 min drain. Dotted line: scheduled scale request. Shading: handoff coordination. Evaluation only; gaps remain unavailable. Both runs start with all twelve high-traffic partitions on Consumer 2.

Figure 5: Scaling with targeted redistribution. Top: completed-message p99 and unfinished evaluation messages at the drain cutoff. Bottom: consumer-position lag during evaluation. Baseline coverage is 99.3%; intervention coverage is 86.0% and 83.3%. Gaps remain unavailable observations.

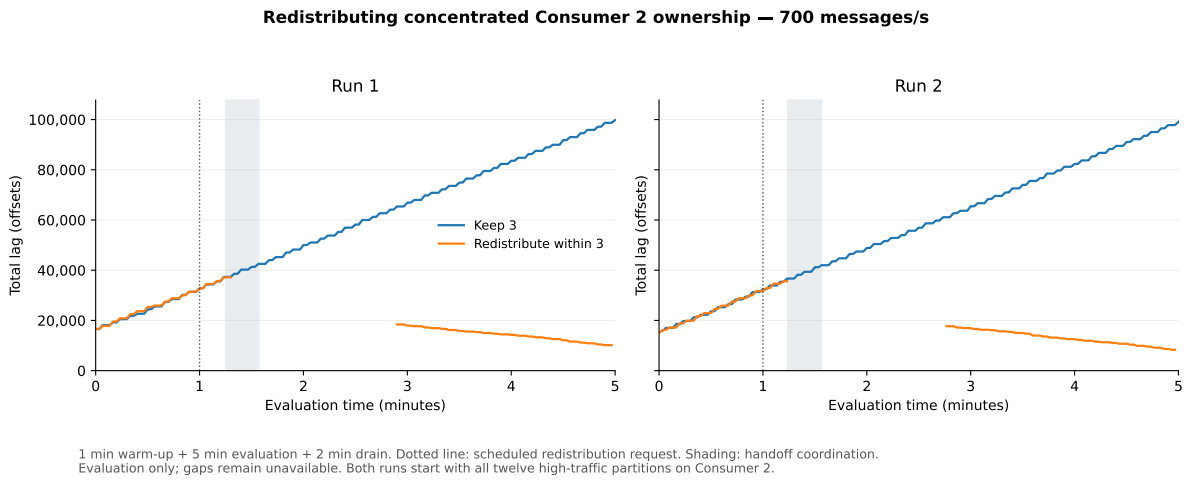
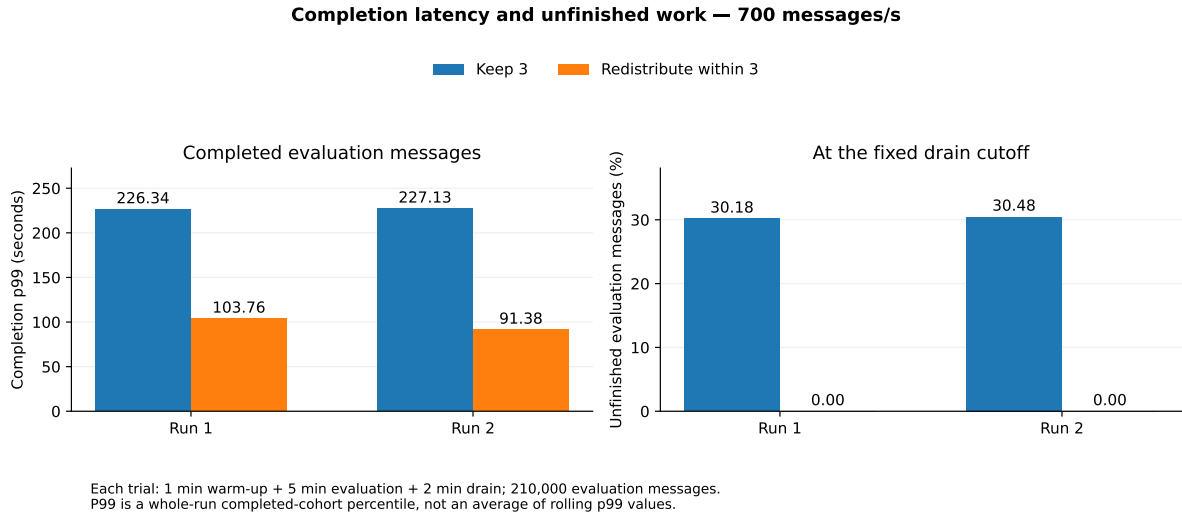


Figure 6: Redistribution within three consumers. Top: completed-message p99 and unfinished evaluation messages at the drain cutoff. Bottom: consumer-position lag during evaluation. Intervention coverage is 67.3% and 68.7%, versus 99.3% in the baselines. These gaps do not represent an equally long processing pause.

3.4 Resource requirements, interruption and recovery

Redistribution improved completion outcomes without increasing requested consumer resources. All four trials in that block used 42.00 core-minutes of requested CPU and 21.00 GiB-minutes of requested memory over the seven-minute evaluation-plus-drain interval, with complete allocation-history coverage. In contrast, the combined interventions requested approximately 76.70 and 76.81 core-minutes and 38.35 and 38.40 GiB-minutes. Table 4 keeps CPU and memory separate and identifies the incomplete resource record in the second combined baseline. These quantities measure requested resources over time, not cloud charges.

Unchanged requests do not imply unchanged resource use. In the redistribution block, observed consumer-process CPU increased from 515.77 to 554.06 core-seconds in Run 1 and from 516.49 to 553.41 core-seconds in Run 2 while more messages completed. The corresponding resident memory accumulated over time changed from 93.71 to 88.36 and from 93.07 to 87.44 GiB-seconds. These process integrals cover approximately 99.52% of evaluation plus drain and exclude unobserved intervals. They describe consumer-process activity and memory footprint, not whole-cluster consumption. CPU, memory, throughput, growth and lag-skew traces are retained in Section 5.

Table 4 covers five minutes of evaluation and two minutes of drain; warm-up is excluded from resource totals. The starred second baseline for scaling with redistribution has only partial resource-history coverage and cannot support a complete-period cost ratio. Missing resource history does not mean zero allocation.

Response	Run	Actual CPU (core-s)	Actual RSS (GiB-min)	Requested CPU (core-min)	Requested memory (GiB-min)
Scaling with targeted redistribution					
Keep 3	1	515.3	1.57	42.00	21.00
Targeted scale 6	1	704.8	2.27	76.70	38.35
Keep 3	2	519.5	1.56	33.75*	16.87*
Targeted scale 6	2	687.6	2.25	76.81	38.40
Redistribution within three consumers					
Keep 3	1	515.8	1.56	42.00	21.00
Redistribute 3	1	554.1	1.47	42.00	21.00
Keep 3	2	516.5	1.55	42.00	21.00
Redistribute 3	2	553.4	1.46	42.00	21.00

Table 4: Consumer resources over five minutes of evaluation and two minutes of drain. Warm-up is excluded.

*Only 80.36% of the requested-resource history was available for this baseline; these partial totals cannot support a full-period cost ratio. The other request histories cover 100%. Actual CPU and RSS totals use observed intervals. RSS is resident set size, the memory held by a consumer process.

3.4.1 Processing interruption and continuing-input recovery

Both interventions temporarily interrupted processing. The combined-action trials had observed processing gaps of 20.20 and 19.99 seconds; redistribution within three consumers had gaps of 15.29 and 15.26 seconds (Table 5). These gaps were checked against recorded processing intervals and handoff events. They are not the same as pod startup time, an ordinary Kafka rebalance duration, or the full interval between the scheduled action and verification of the new assignment. Approximate cross-host timing remains subject to the recorded clock-probe uncertainty.

Messages continued to arrive during these interruptions. The event records show increases of 14,143 and 13,992 acknowledged but unfinished messages during the combined-action gaps, and 10,700 and 10,682 during the redistribution-only gaps. Figure 7 reconstructs unfinished work from message identifiers and acknowledgment and completion times, including warm-up messages. This provides evidence during invalid offset-monitoring intervals. It is a different measurement from consumer-position lag and does not by itself show how many additional unfinished messages the intervention caused compared with running without an intervention.

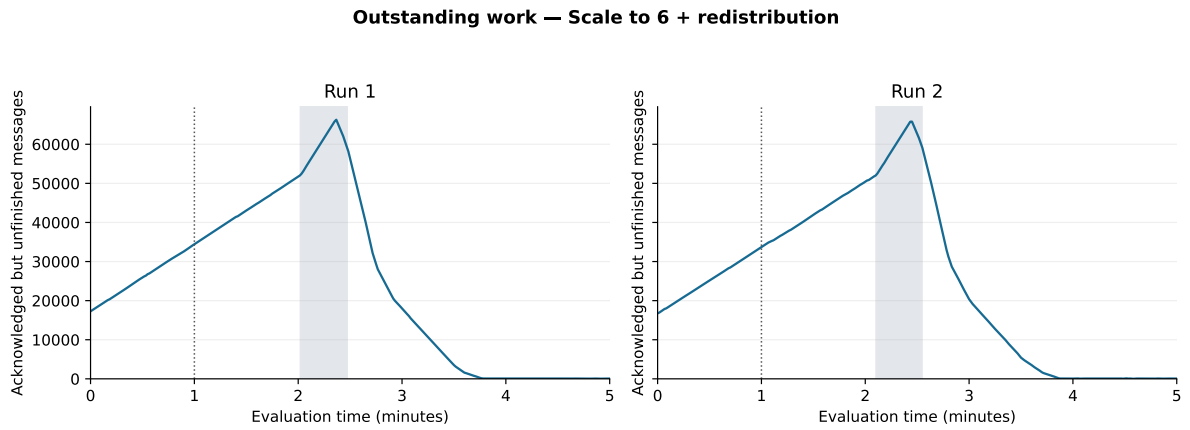
In Table 5, active ownership means that the new assignment has been verified. Processing may resume before this check finishes, so the reported intervals have different endpoints and are not interchangeable.

Intervention	Processing gap (s)	Release to active (s)	Scheduled to active (s)	Unfinished increase during gap
Combined, Run 1	20.20	27.03	88.28	14,143
Combined, Run 2	19.99	26.55	92.79	13,992
Redistribution, Run 1	15.29	19.29	34.26	10,700
Redistribution, Run 2	15.26	19.38	33.78	10,682

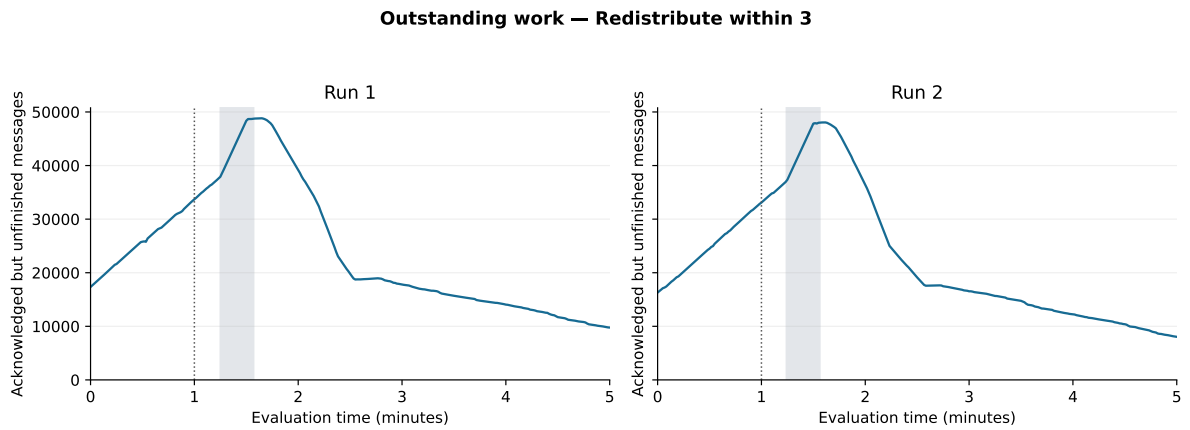
Table 5: Handover times and additional unfinished work.

Recovery while input continues is a stricter outcome than completing all evaluation messages after producers stop. For these blocks, the diagnostic requires total processing backlog to remain at or below 100 offsets for 30 consecutive seconds, with valid observations from the same partition owners and no gaps exceeding three seconds. In the combined block, recovery was confirmed approximately 120.4 and 117.7 seconds after resume. That recovery check was defined after the combined-action trials. The same criterion was fixed before the redistribution-only block, where neither trial confirmed recovery before production ended; thresholds of 50 and 200 offsets gave the same unconfirmed result.

The redistribution trials were nevertheless catching up near the end of evaluation. Over the final two minutes, covered processing-backlog growth was approximately -65.7 and -72.9 offsets/s, but the last valid backlogs were still 10,166 and 8,326 offsets. The subsequent drain allowed all evaluation messages to finish. Thus, the observations support improved processing and less unfinished work at the cutoff, while leaving open whether the unchanged input could be sustained with a sufficiently small backlog over a longer period.



Reconstructed from message identities and timestamps, including warm-up. This is not broker offset lag.
Shading: release requested to active verified. Dotted line: scheduled intervention.



Reconstructed from message identities and timestamps, including warm-up. This is not broker offset lag.
Shading: release requested to active verified. Dotted line: scheduled intervention.

Figure 7: Acknowledged but unfinished messages around the intervention. Top: scaling with targeted redistribution. Bottom: redistribution within three consumers. Counts are reconstructed from message events and include warm-up work. They are distinct from offset-based lag and evaluation-only unfinished messages at drain.

4 Supporting Ownership Diagnostics

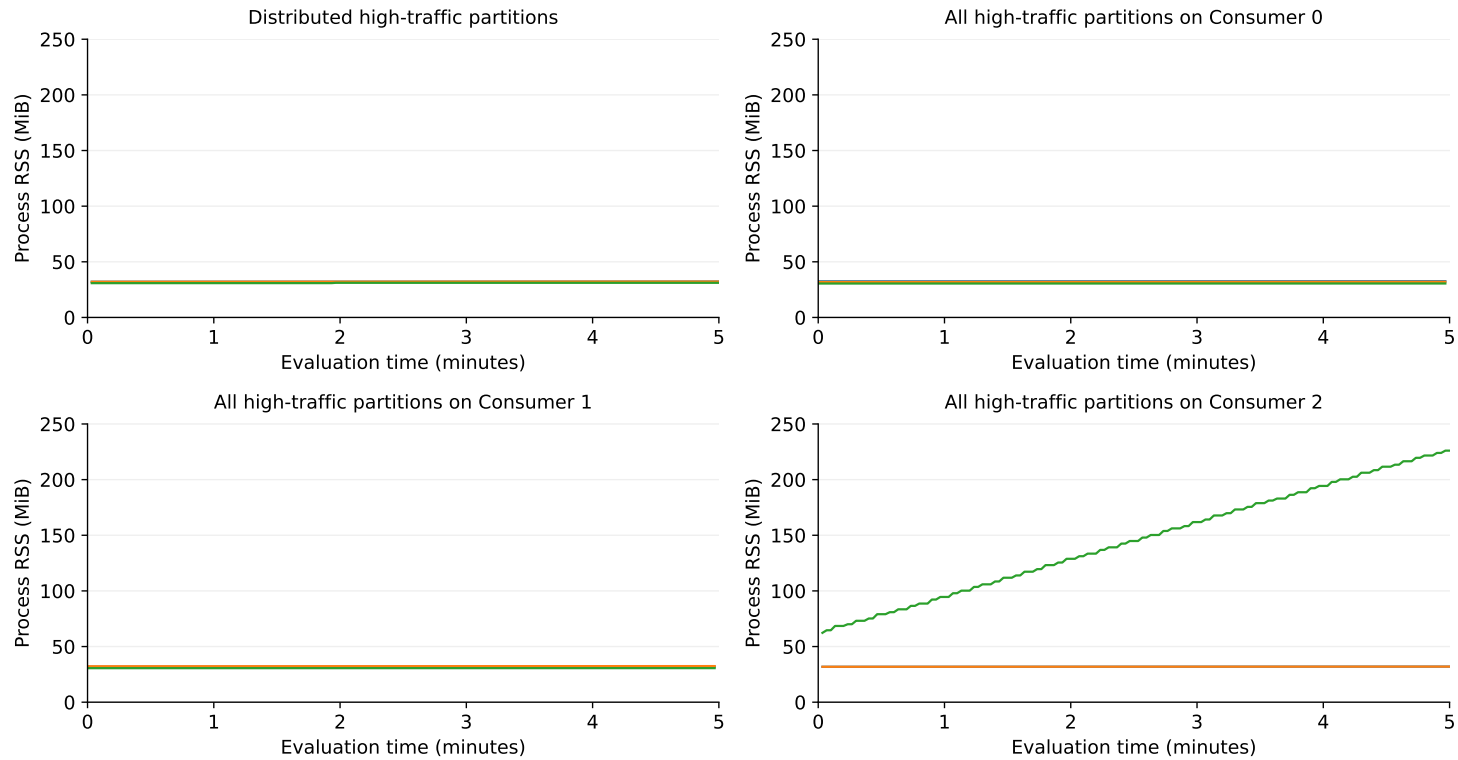
Figures 8–10 complete the seven metric comparisons for the four 700-message/s ownership trials. All show only the five-minute evaluation interval, with distributed ownership and concentration on Consumers 0, 1 and 2 in the same display order. Each full trial includes one minute of warm-up and two minutes of drain. Display order is different from the actual execution order (distributed, C0, C2, C1).

Consumer 2’s mean resident memory, the physical memory occupied by its process, reached 145.4 MiB in its concentrated trial, compared with approximately 31.9 MiB for the other consumers in that trial. The cause of this process-memory difference was not identified. The measurement does not represent all memory used on the node. The mean lag-skew ratios across valid observations were 9.241, 9.252, 10.004 and 4.927 in display order. These are means of valid maximum/mean partition-lag ratios, not ratios derived from aggregate lag means. The lower ratio in the overloaded case illustrates the need to interpret relative skew together with backlog magnitude and growth.

Consumer memory usage

700 messages/s total · Three consumers · 80% of input across 12 of 60 partitions

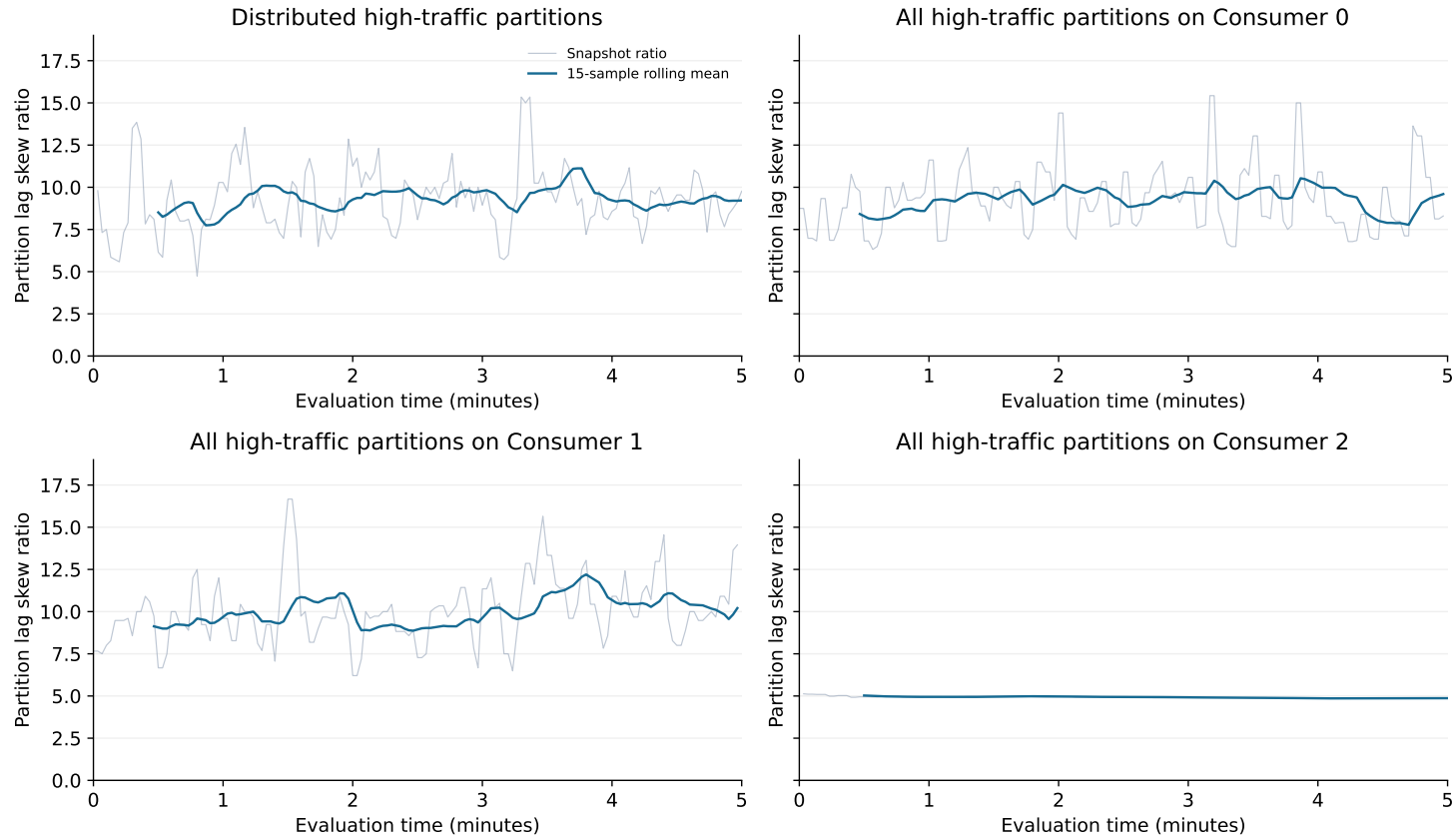
Consumer 0 Consumer 1 Consumer 2



Five-minute evaluation. Main panels share the same vertical scale.
Memory is process resident set size (RSS).

Figure 8: Consumer-process resident memory (RSS) during evaluation. Panels share a scale. The concentrated Consumer 2 trial shows increased process memory, but the source of the allocation is not isolated. These measurements exclude other processes, brokers and node-wide memory use.

Partition lag skew — 700 messages/s, three consumers



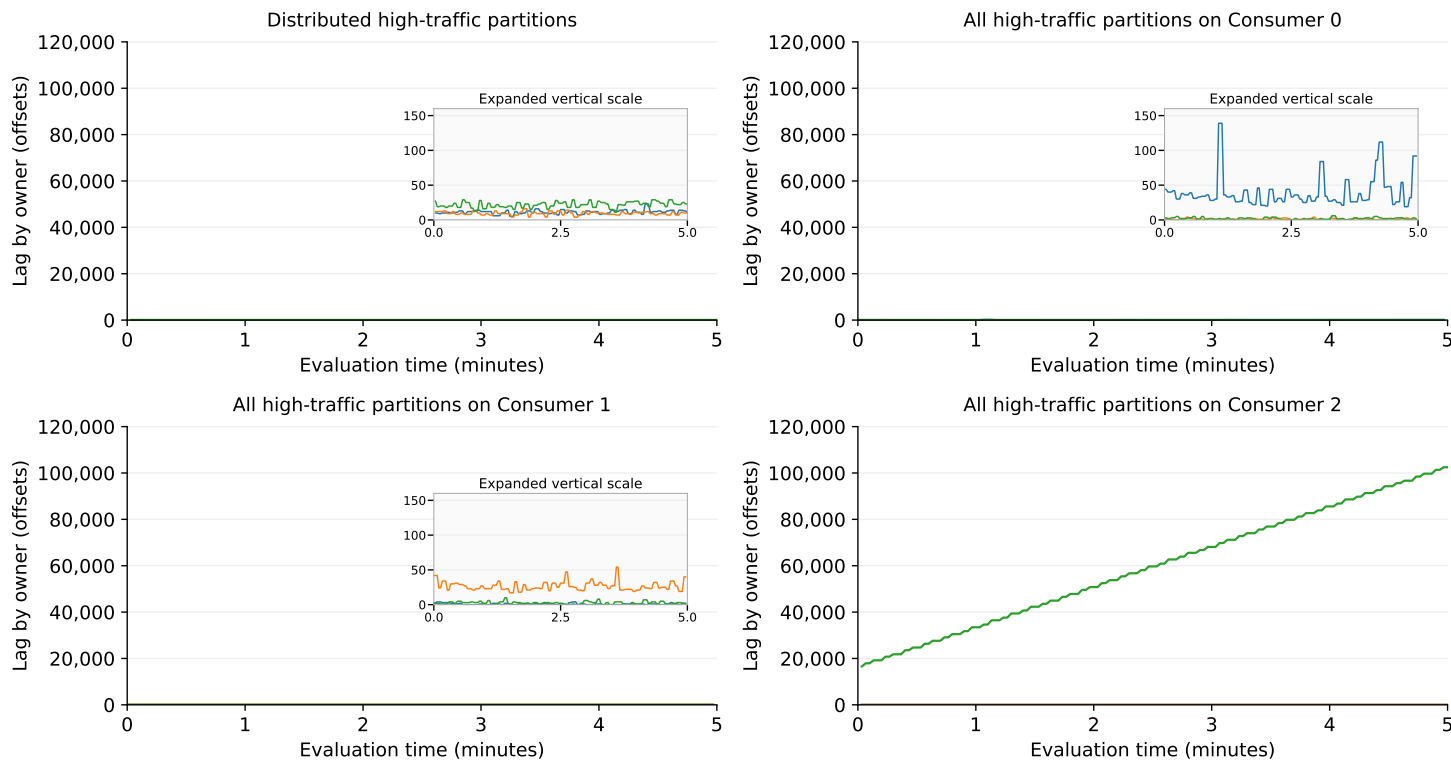
Ratio = maximum partition lag / mean partition lag; zero if all lag is zero. Rolling mean: 15 consecutive valid samples.

Figure 9: Partition lag-skew ratio, defined as maximum partition lag divided by mean partition lag. A high ratio can occur when absolute lag is small. The Consumer 2 concentration trial has substantial lag across multiple high-traffic partitions and a lower mean ratio than the low-lag layouts.

Lag by consumer

700 messages/s total · Three consumers · 80% of input across 12 of 60 partitions

— Consumer 0 — Consumer 1 — Consumer 2



Five-minute evaluation. Main panels share the same vertical scale. Insets show the same data with a shared expanded scale.

Figure 10: Consumer-position lag grouped by the fixed partition owner. Main panels share a scale, while labelled insets expand the small fluctuations. The large lag in the Consumer 2 concentration trial is associated with its assigned partitions. These are fixed initial assignments, not before/after observations of live redistribution.

5 Supporting Measurements for Live Interventions

The first set of figures retains the per-consumer and time-dependent measurements for the two earlier live-intervention blocks. Each figure compares separate eight-minute trials, comprising one minute of warm-up, five minutes of evaluation and two minutes of drain. The plotted interval is the five-minute evaluation unless stated otherwise. Request markers and shaded coordination intervals describe recorded intervention events; blank intervals remain unavailable observations.

5.1 Monitoring coverage and interpretation

Table 6 reports consumer-position lag only over valid monitoring intervals. The baseline series cover approximately 99.3% of evaluation, whereas the intervention series lose coverage around ownership transfer. In particular, the redistribution trials contain intervals without valid lag measurements of approximately evaluation seconds 80–172 and 76–164. Their duration does not equal the approximately 15.3-second verified processing interruption. Neither the sampled maximum nor the mean establishes the true full-transition backlog when observations are missing. The event-derived outstanding-work figures in Section 3.4 provide complementary evidence without filling these gaps.

AWS documents a related monitoring limitation in Amazon MSK: consumer-lag metrics are emitted only for groups in the **STABLE** or **EMPTY** states and are absent while a group is unstable [1]. This concerns missing measurements, not lost records, and does not establish the cause of gaps in this pipeline.

The five-minute trials used an earlier monitor that sometimes continued to report unavailable lag after processing resumed. The client did not report a position for a newly assigned partition until that partition returned its first record. The revised monitor temporarily uses the verified starting position during this interval. It still requires valid broker and processing positions and exactly one assigned consumer per partition, and stops using the temporary position once records arrive or ownership changes. Total lag is reported only when every partition has a valid measurement. Actual transfer intervals remain unavailable, and missing measurements from the earlier trials are not reconstructed. The technical check of this correction is excluded from the 36 performance trials.

Table 6 summarizes consumer-position lag during the five-minute evaluation. Means use valid observed intervals and maxima use valid samples. All intervention series fall below the 90% monitoring-coverage requirement for aggregate backlog comparisons, so these summaries are descriptive and do not support full-period lag-reduction percentages. Latency and unfinished-message outcomes use separate message records.

Response	Run	Mean total lag	Maximum total backlog	Lag coverage (%)	Maximum partition lag	Mean skew
Scaling with targeted redistribution						
Keep 3	1	58,517	101,938	99.3	8,290	4.92
Targeted scale 6	1	19,438	50,671	86.0	4,960	8.47
Keep 3	2	57,077	99,351	99.3	8,047	4.93
Targeted scale 6	2	19,473	51,721	83.3	4,245	8.92
Redistribution within three consumers						
Keep 3	1	58,189	99,805	99.3	8,109	4.93
Redistribute 3	1	19,243	37,159	67.3	3,724	9.41
Keep 3	2	57,329	99,153	99.3	8,025	4.92
Redistribute 3	2	17,609	35,615	68.7	3,581	9.57

Table 6: Five-minute lag measurements over valid evaluation observations. Missing intervals are excluded, not treated as zero.

Processing-backlog growth and relative lag skew answer different questions. Growth indicates whether queued work is increasing or decreasing; skew indicates how unevenly the measured lag is distributed. A flat positive growth curve indicates continuing accumulation. A high skew ratio can coexist with a small absolute backlog. The redistribution trials illustrate another distinction: backlog growth calculated from the valid intervals across the evaluation remained positive, approximately +61.0 and +54.7 offsets/s, while growth over the final two minutes was negative. The first summary includes accumulation before the intervention and excludes intervals without valid measurements. It is therefore not simply the difference between the first and last backlog values divided by five minutes.

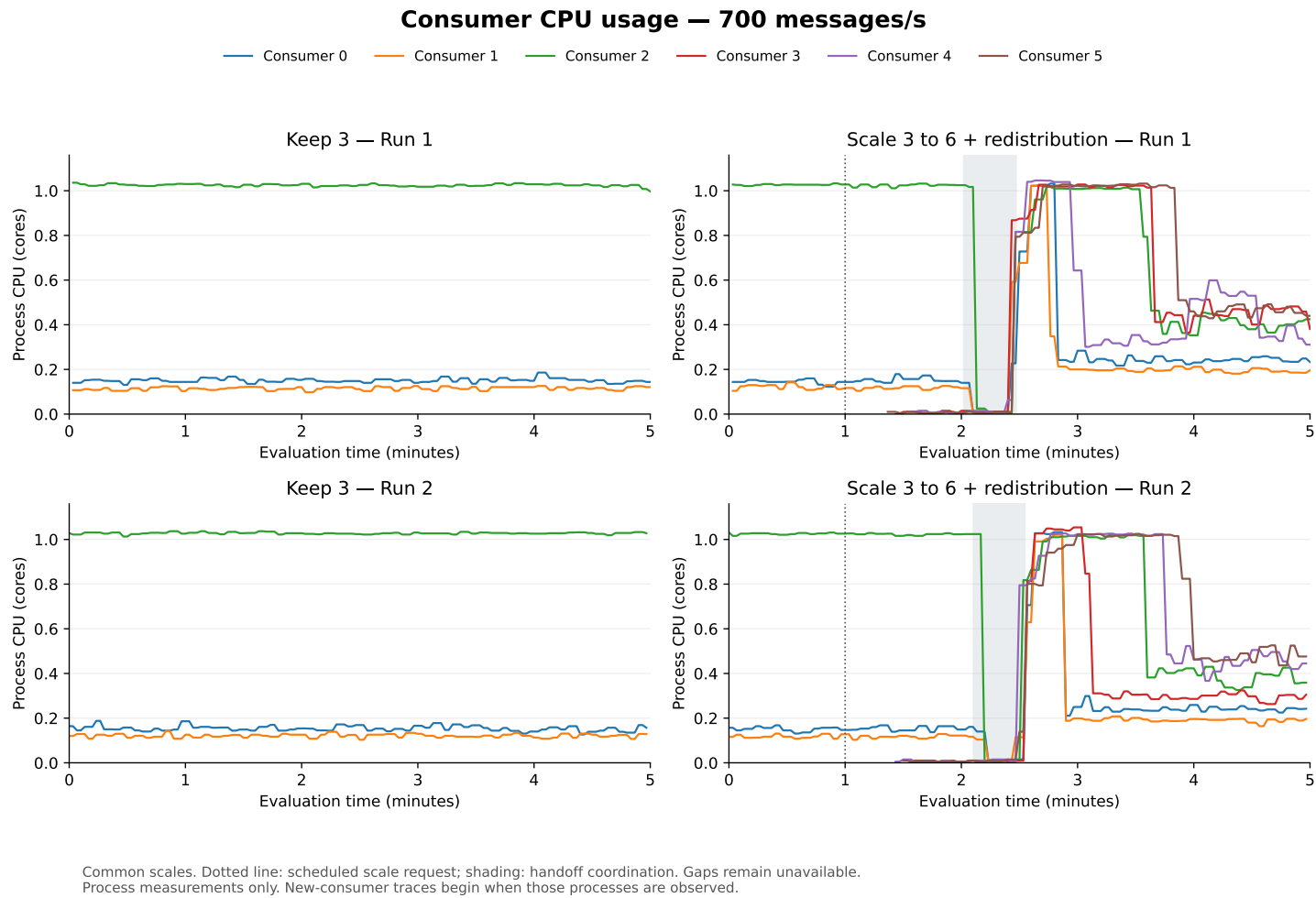


Figure 11: Adding consumers with explicit redistribution: consumer-process CPU use. CPU use is expressed in cores; one core corresponds to 100% process CPU. It is distinct from requested CPU and excludes whole-node, broker, and producer use. Each panel covers five minutes of evaluation within an eight-minute trial.

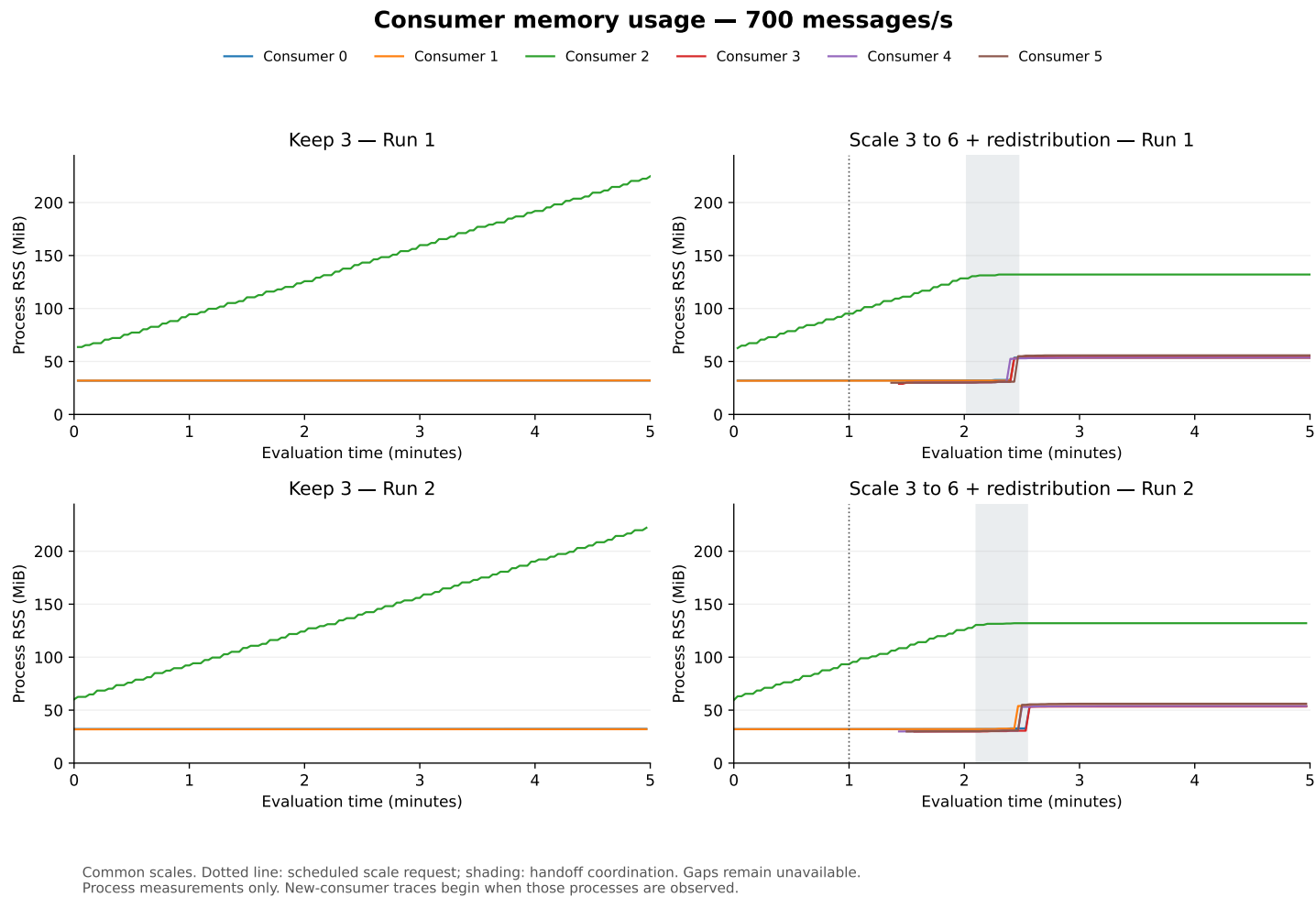
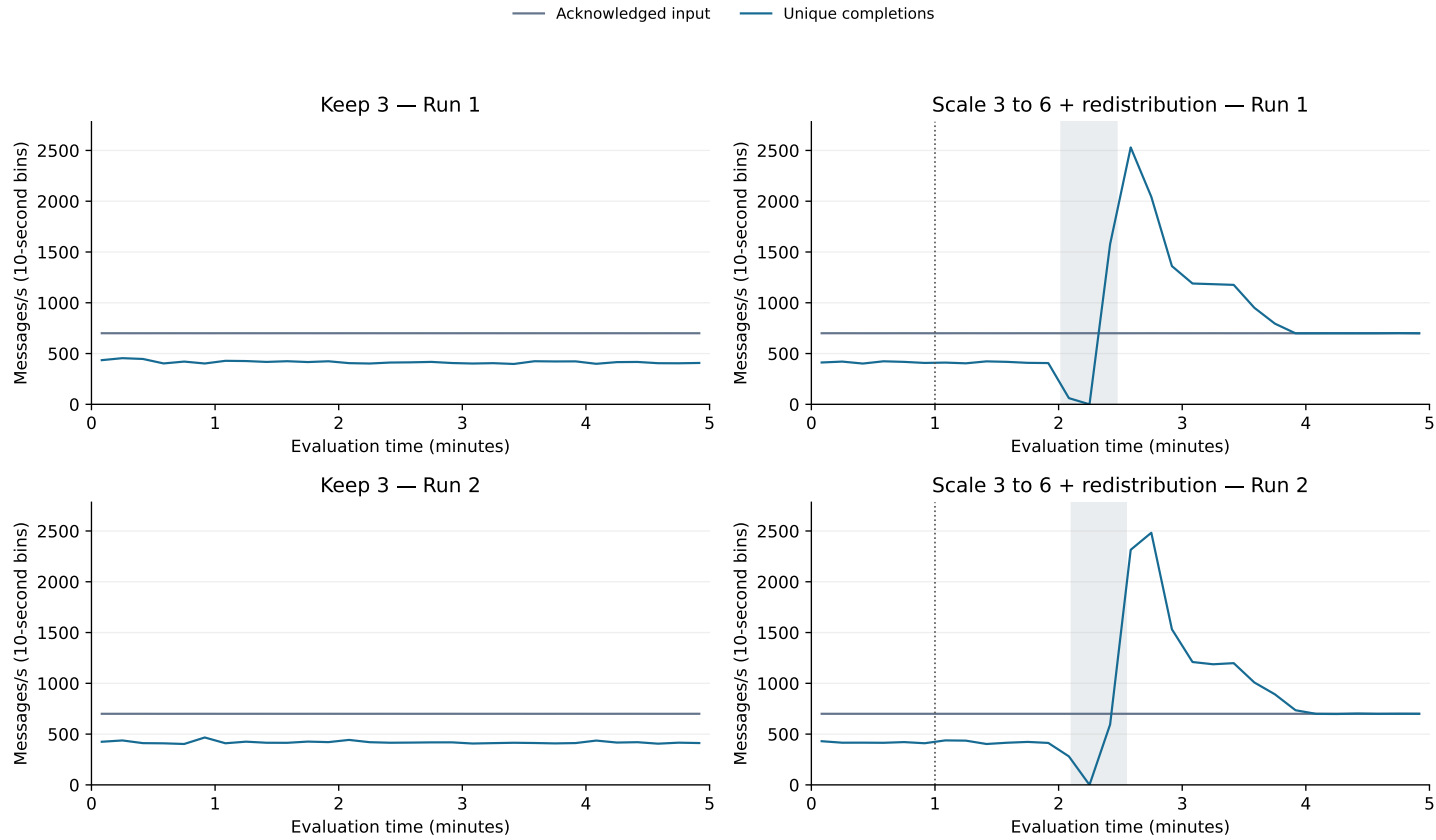


Figure 12: Adding consumers with explicit redistribution: consumer-process resident memory. Resident set size measures the memory held by each consumer process. It is not the Kubernetes memory request, a transfer-volume measurement or a monetary cost. Each panel covers five minutes of evaluation within an eight-minute trial.

Input and completion throughput — 700 messages/s



Common scales. Dotted line: scheduled scale request; shading: handoff coordination. Gaps remain unavailable. Completions include warm-up records finishing during evaluation.

Figure 13: Adding consumers with explicit redistribution: acknowledged input and distinct completion throughput. Ten-second bins show input and processing progress. Completions can include queued warm-up messages, so catch-up throughput can exceed the 700-message/s input rate. Each panel covers five minutes of evaluation within an eight-minute trial.

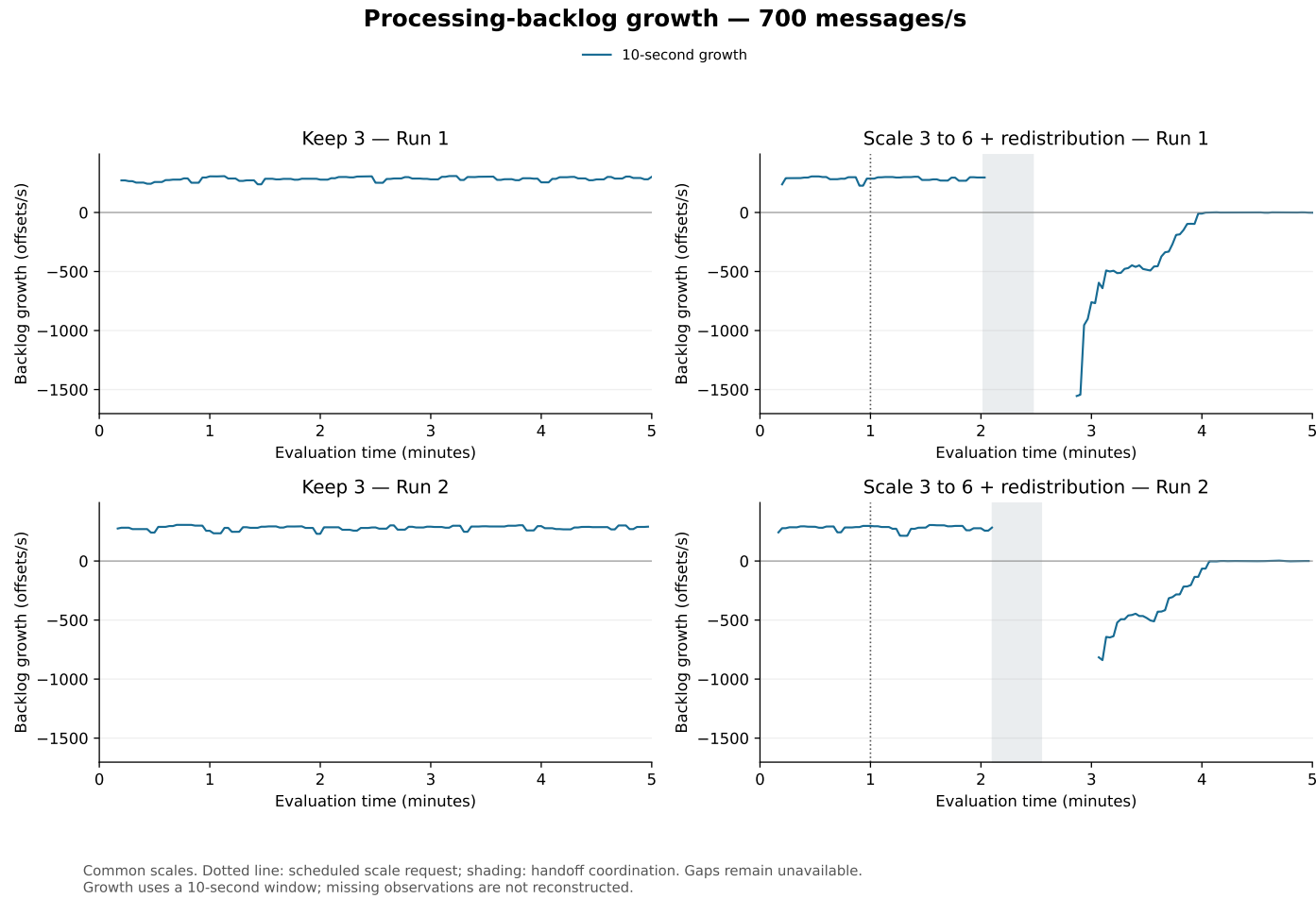


Figure 14: Adding consumers with explicit redistribution: processing-backlog growth. Growth uses the position up to which all preceding messages have completed, measured over a ten-second window, shortened from thirty seconds after the trials and applied consistently across this block. Negative values indicate catch-up, not necessarily an empty backlog. Invalid intervals and ownership or offset resets are not bridged; a complete valid window is required before growth resumes. The separate requirement for thirty seconds of low backlog before confirming recovery is unchanged. Each panel covers five minutes of evaluation within an eight-minute trial.

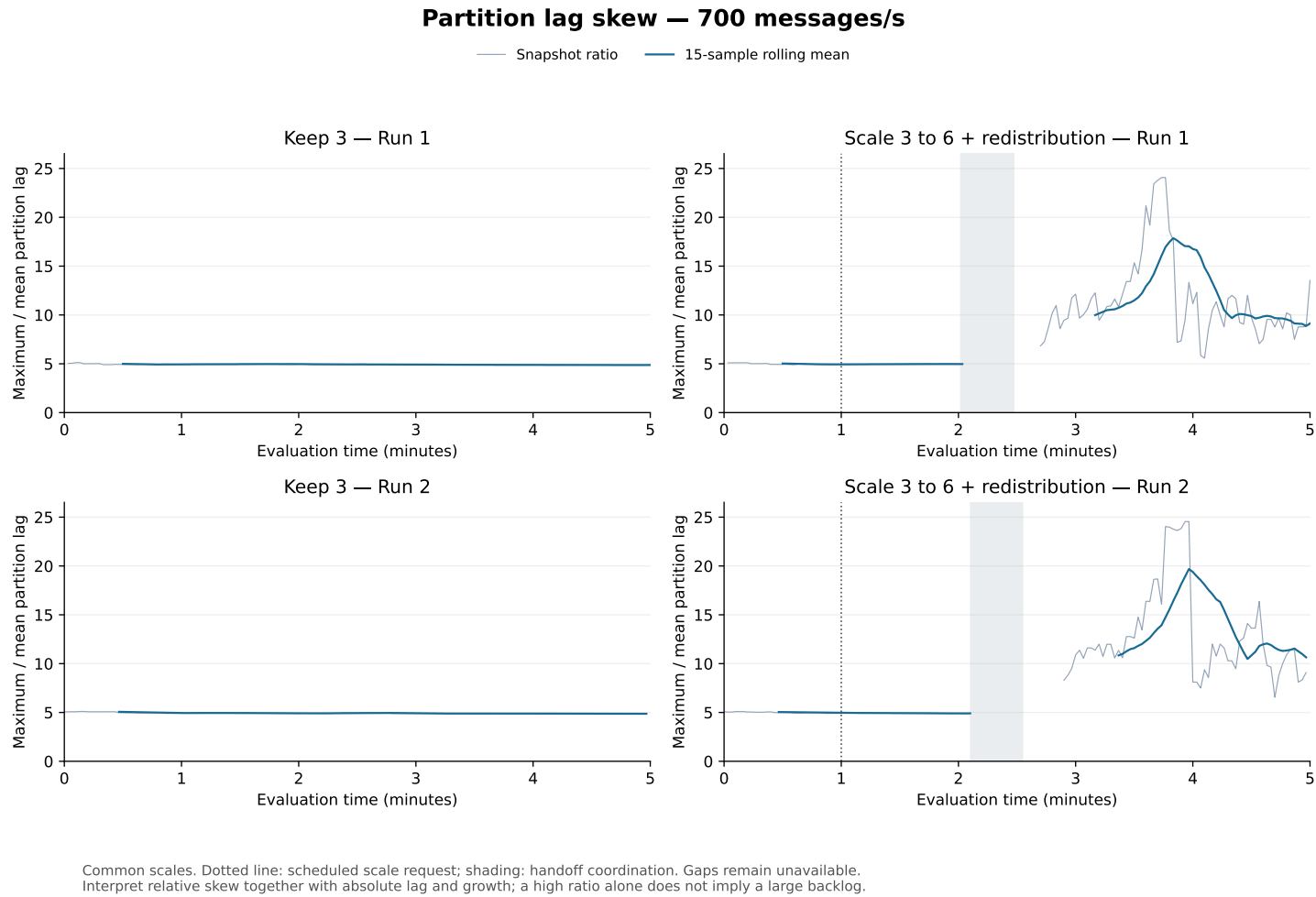
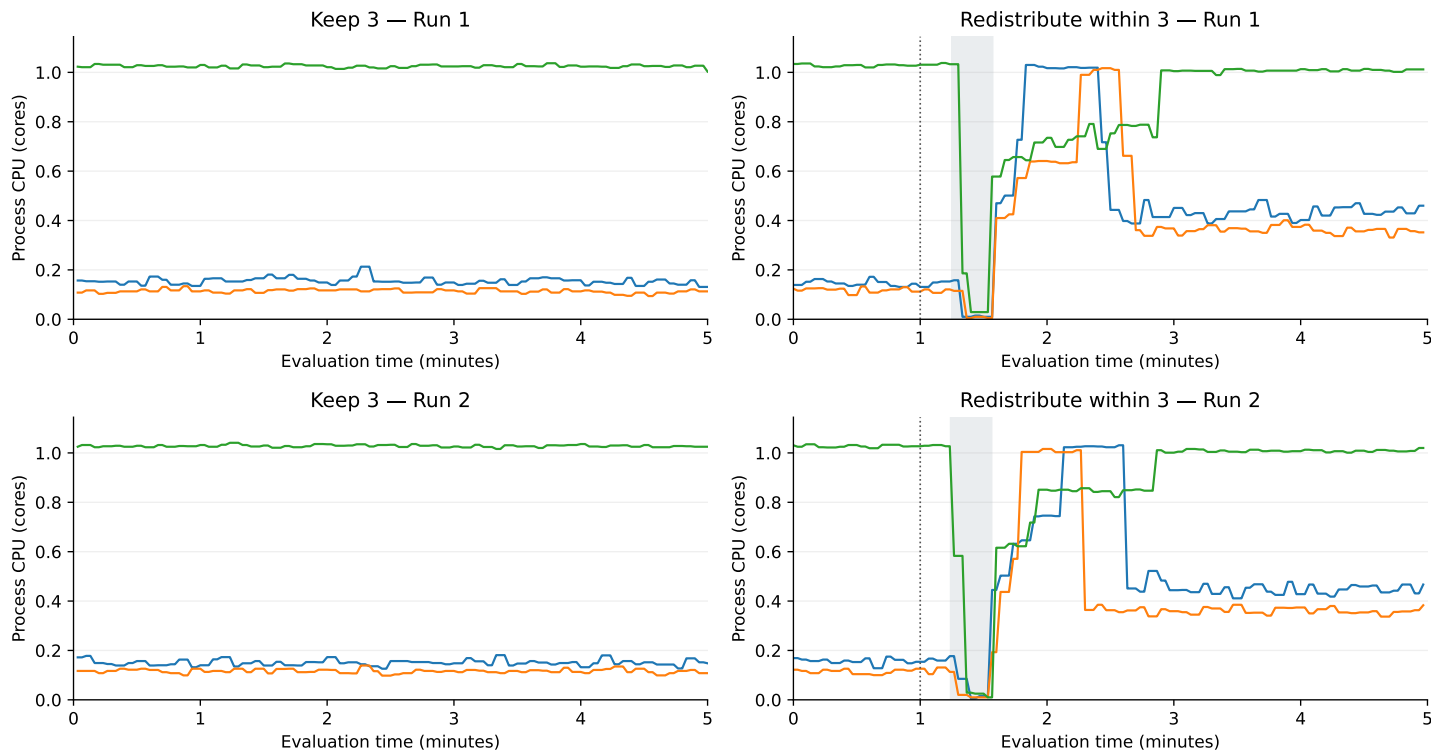


Figure 15: Adding consumers with explicit redistribution: partition-level lag skew. The maximum-to-mean ratio is interpreted with absolute lag and growth. A high relative ratio alone does not establish severe overload; a ratio near one can also coexist with broadly distributed backlog. Each panel covers five minutes of evaluation within an eight-minute trial.

Consumer CPU usage — 700 messages/s

— Consumer 0 — Consumer 1 — Consumer 2



Common scales. Dotted line: scheduled redistribution request; shading: handoff coordination. Gaps remain unavailable. Process measurements only. The same three consumers run throughout each trial.

Figure 16: Redistribution within three consumers: consumer-process CPU use. CPU use is expressed in cores; one core corresponds to 100% process CPU. It is distinct from requested CPU and excludes whole-node, broker and producer use. Each panel covers five minutes of evaluation within an eight-minute trial.

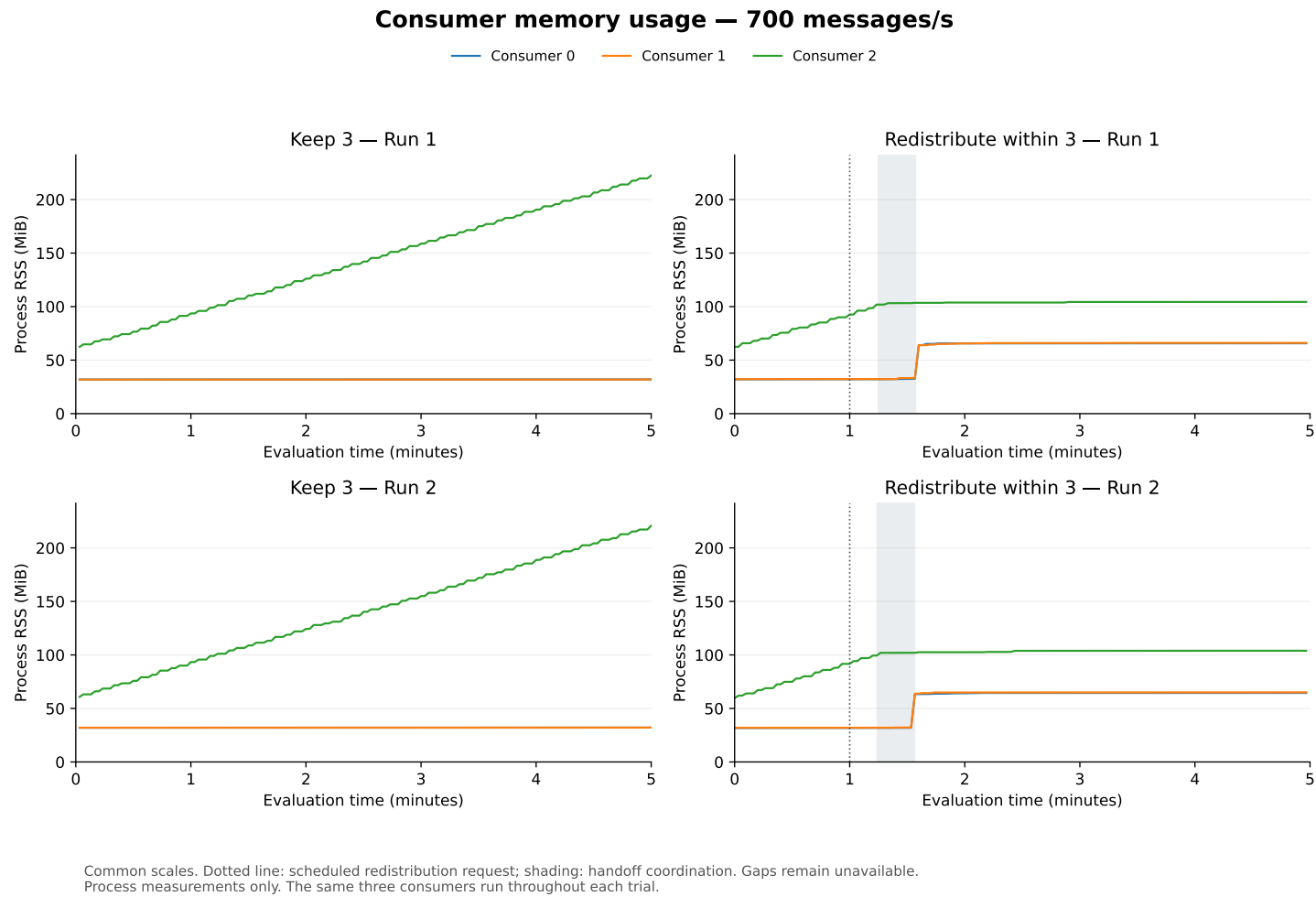


Figure 17: Redistribution within three consumers: consumer-process resident memory. Resident set size measures the memory held by each consumer process. It is not the Kubernetes memory request, a transfer-volume measurement or a monetary cost. Each panel covers five minutes of evaluation within an eight-minute trial.

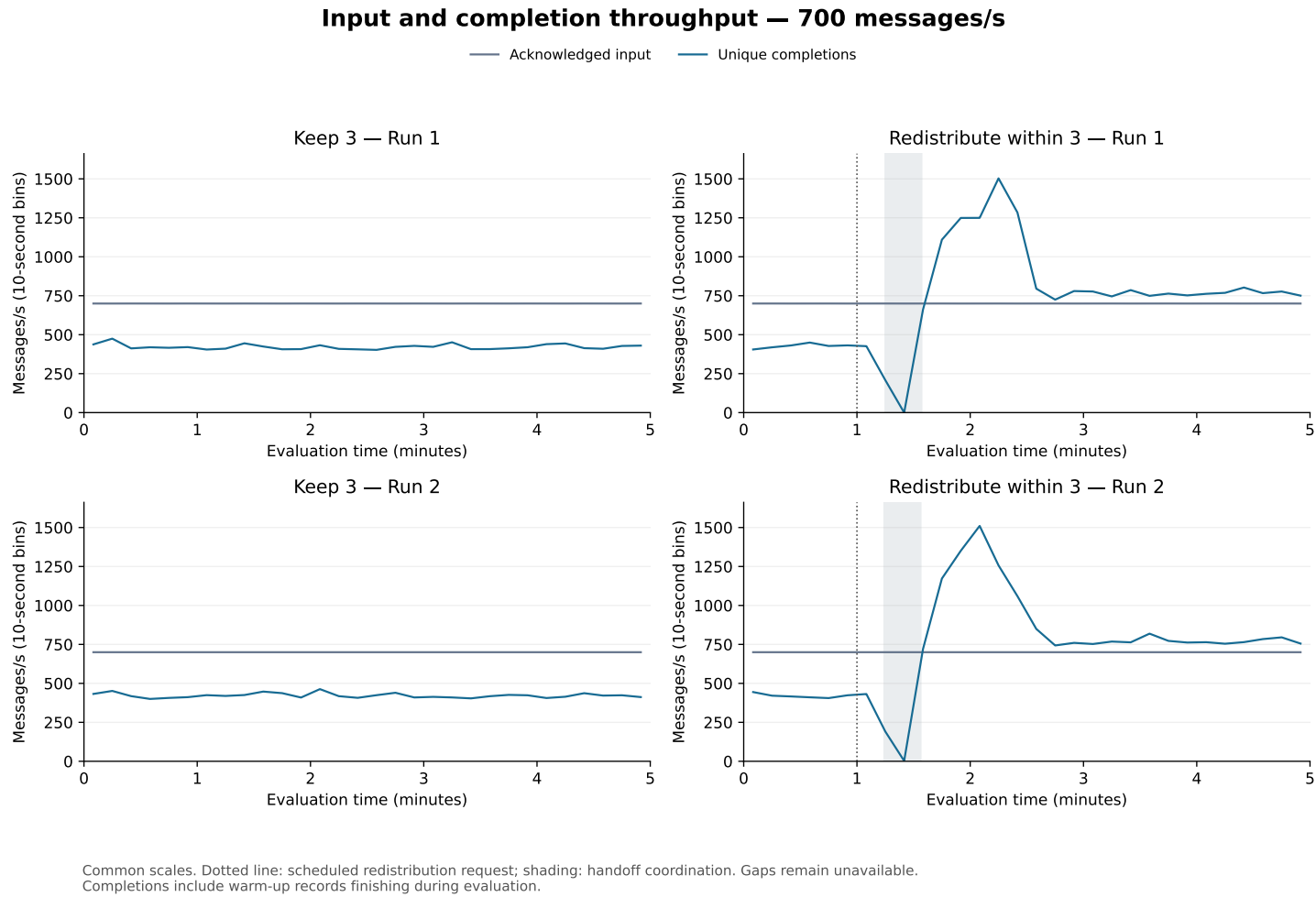
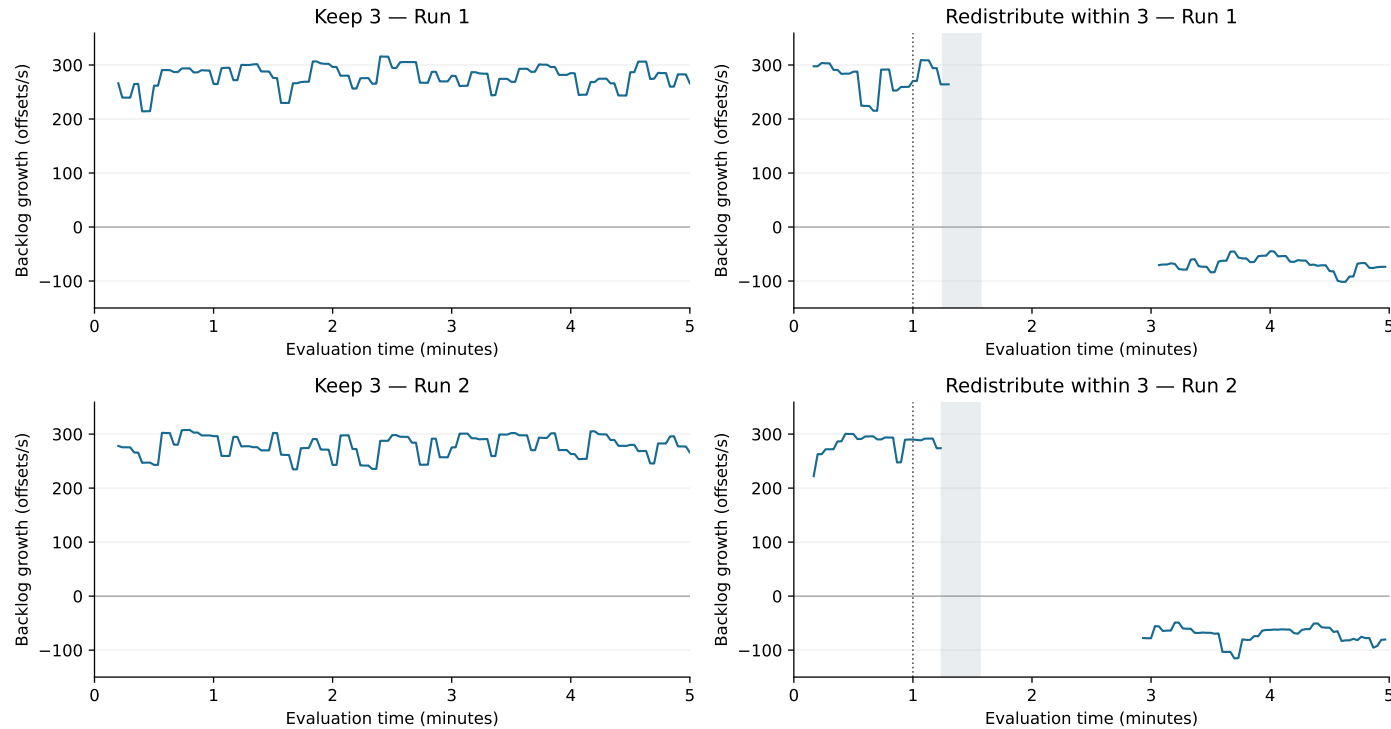


Figure 18: Redistribution within three consumers: acknowledged input and distinct completion throughput. Ten-second bins show input and processing progress. Completions can include queued warm-up messages, so catch-up throughput can exceed the 700-message/s input rate. Each panel covers five minutes of evaluation within an eight-minute trial.

Processing-backlog growth — 700 messages/s

— 10-second growth



Common scales. Dotted line: scheduled redistribution request; shading: handoff coordination. Gaps remain unavailable. Growth uses a 10-second window; missing observations are not reconstructed.

Figure 19: Redistribution within three consumers: processing-backlog growth. Growth uses the position up to which all preceding messages have completed, measured over a ten-second window, shortened from thirty seconds after the trials and applied consistently across this block. Negative values indicate catch-up, not necessarily an empty backlog. Invalid intervals and ownership or offset resets are not bridged; a complete valid window is required before growth resumes. The separate requirement for thirty seconds of low backlog before confirming recovery is unchanged. Each panel covers five minutes of evaluation within an eight-minute trial.

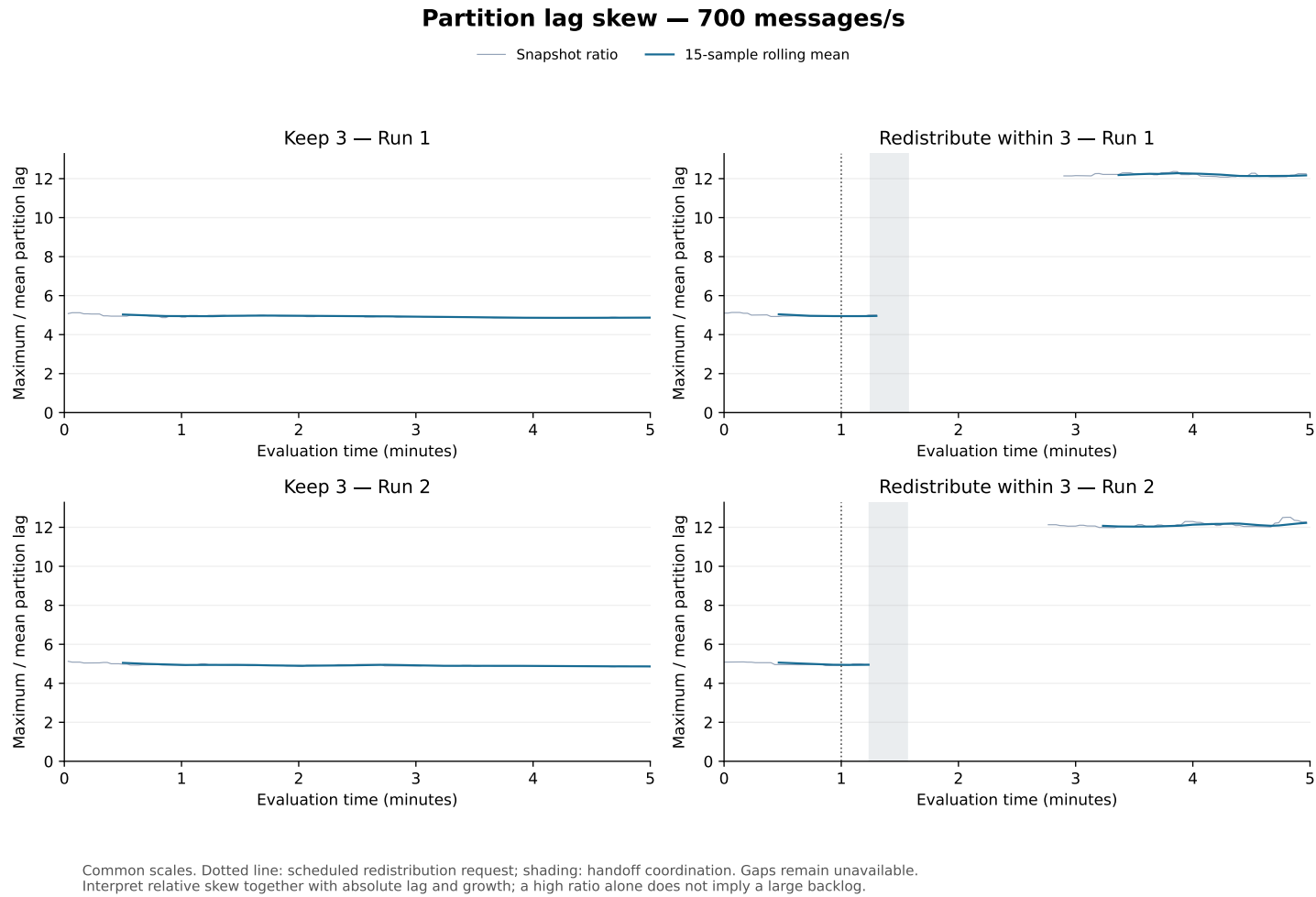
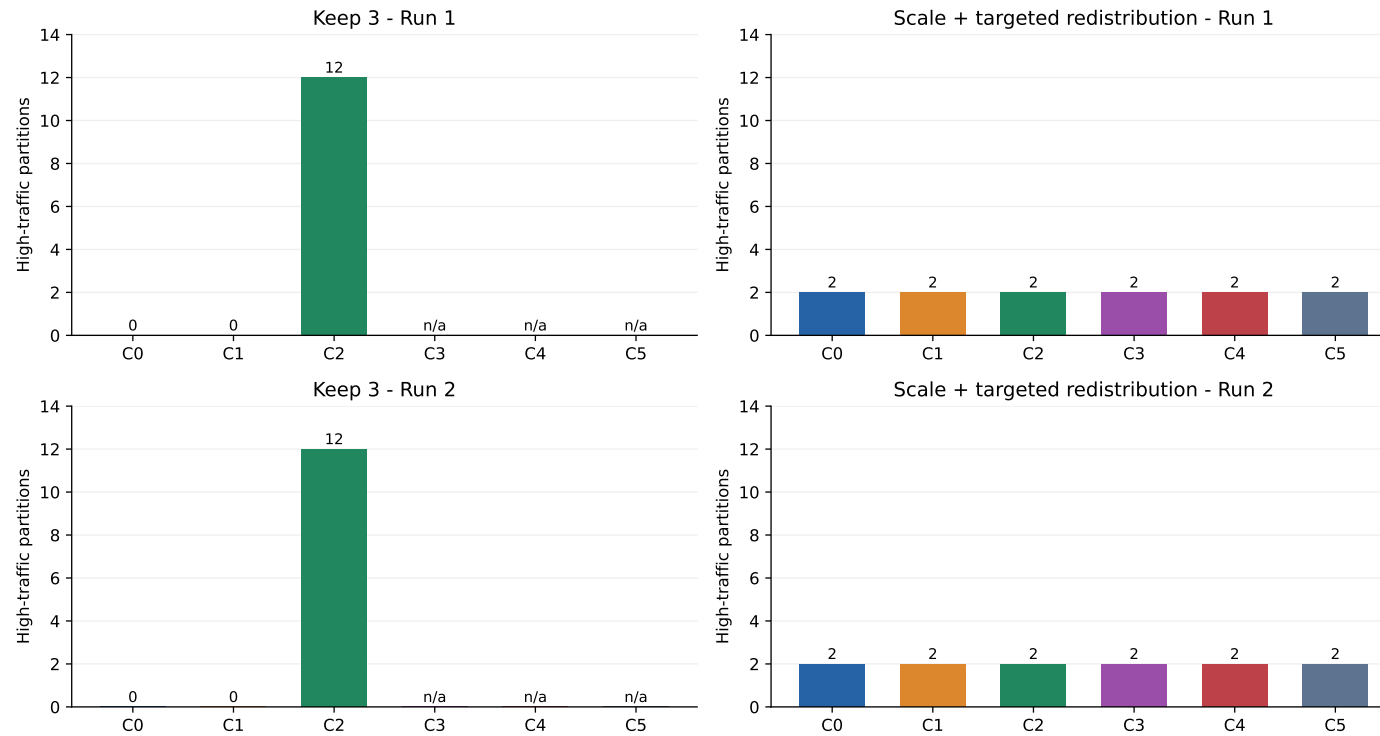


Figure 20: Redistribution within three consumers: partition-level lag skew. The maximum-to-mean ratio is interpreted with absolute lag and growth. A high relative ratio alone does not establish severe overload; a ratio near one can also coexist with broadly distributed backlog. Each panel covers five minutes of evaluation within an eight-minute trial.

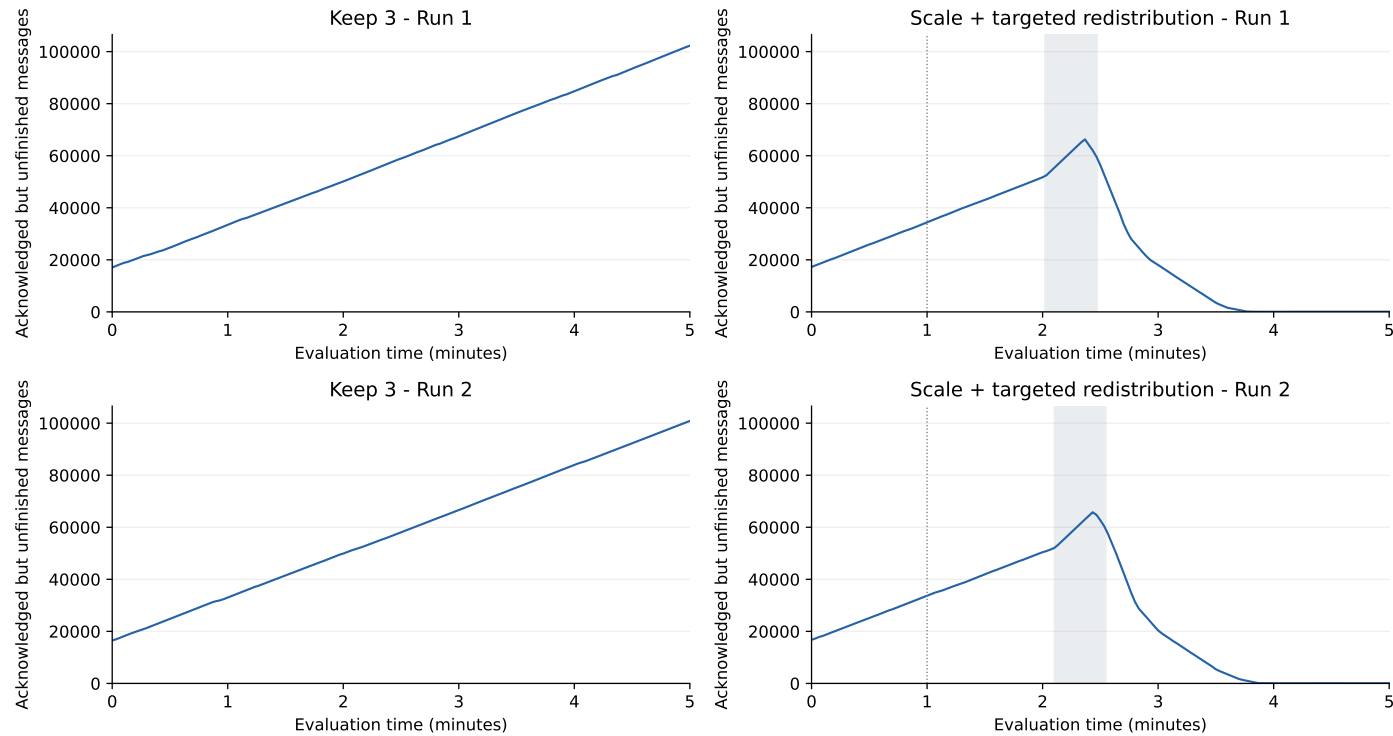
Observed high-traffic partition ownership at the end of evaluation - 700 messages/s



Each trial: 1 min warm-up + 5 min evaluation + 2 min drain. Common scales.
 Last complete valid ownership snapshot; n/a means no partition ownership in that snapshot.
 C0-C5 identify consumers. Equal partition counts do not imply equal processing load.

Figure 21: Observed high-traffic partition ownership in the five-minute scaling-with-redistribution trials. Baselines retained all twelve on Consumer 2; targeted scaling placed two on each of six consumers. Equal partition counts do not establish equal load.

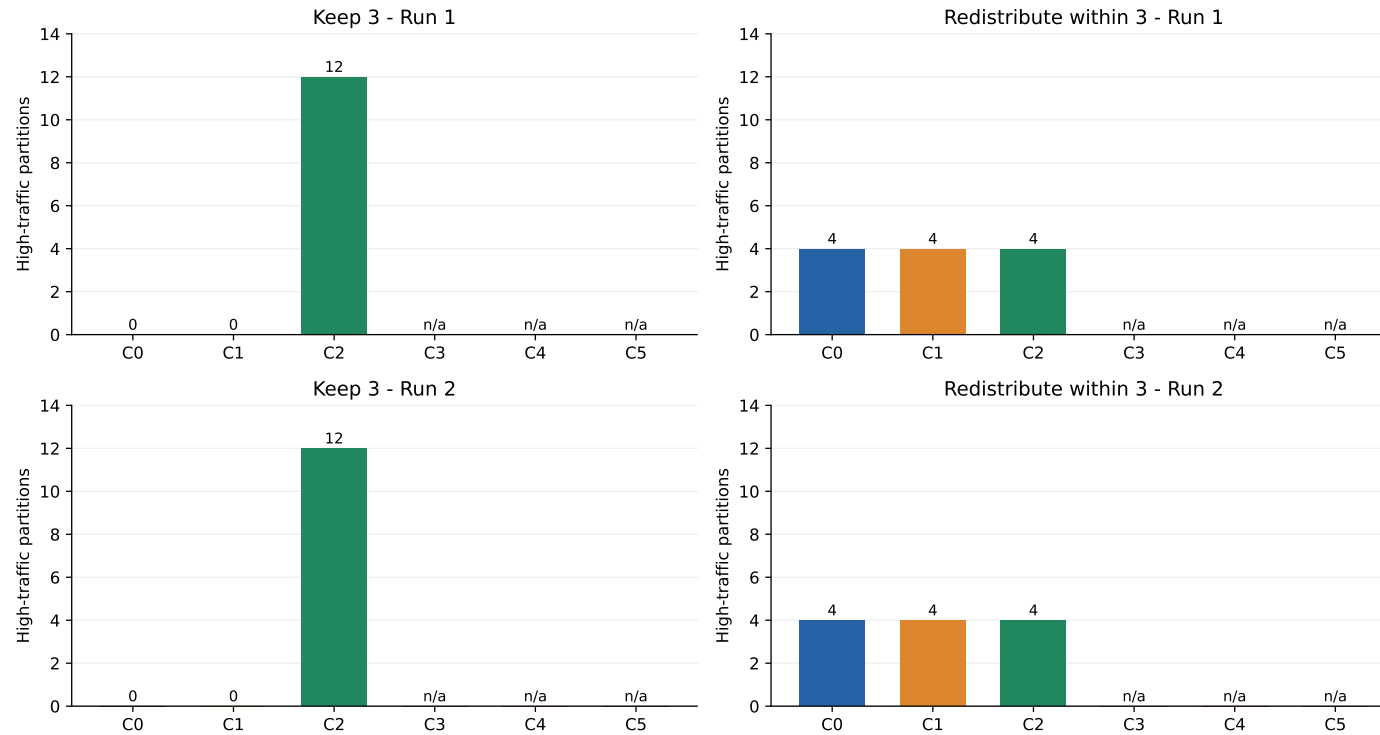
Acknowledged but unfinished work - 700 messages/s



Each trial: 1 min warm-up + 5 min evaluation + 2 min drain. Common scales.
 Distinct ACK/completion events, including warm-up; this is not broker offset lag.
 Dotted line: scheduled intervention. Shading: release requested to active verified.

Figure 22: Distinct acknowledged messages not yet completed during the five-minute scaling-with-redistribution trials, including warm-up work. The dotted line marks the scheduled intervention; shading spans release request to verified active ownership. Each full trial lasts eight minutes including warm-up and drain.

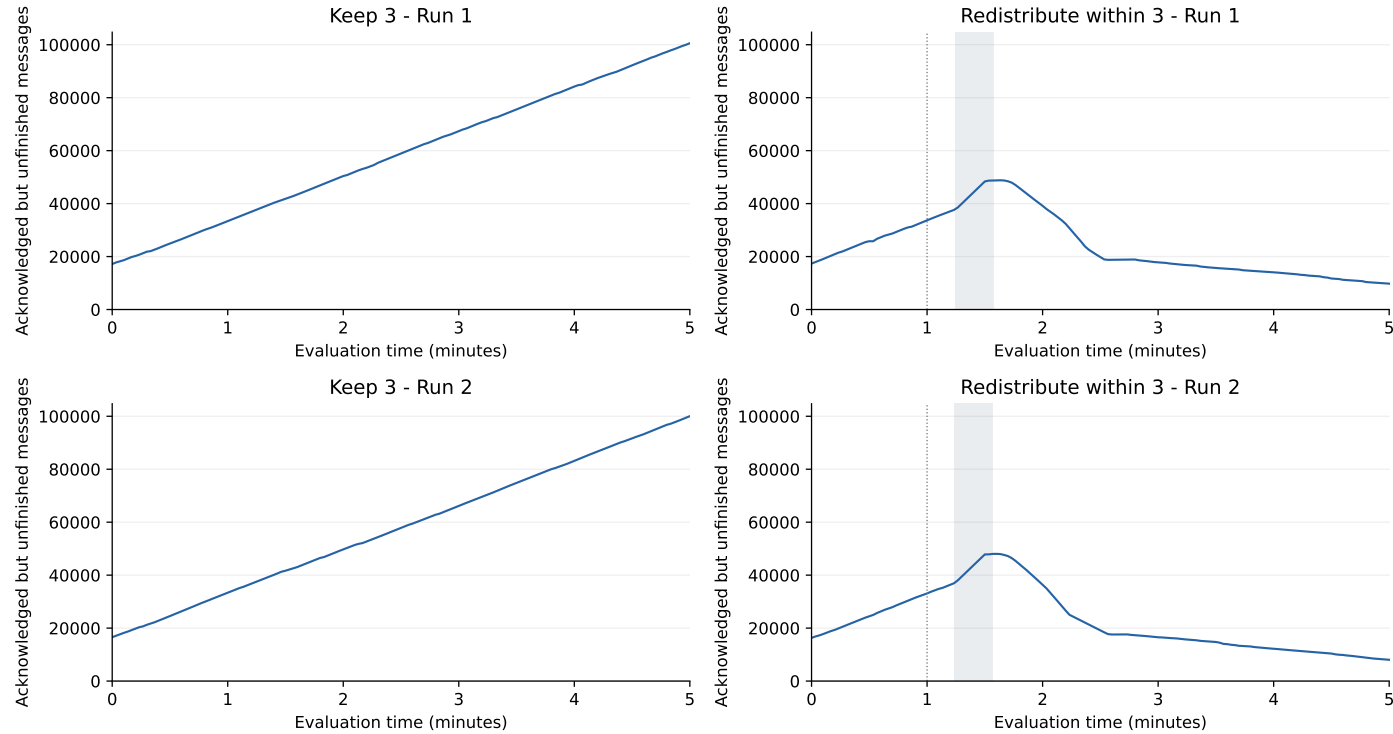
Observed high-traffic partition ownership at the end of evaluation - 700 messages/s



Each trial: 1 min warm-up + 5 min evaluation + 2 min drain. Common scales.
 Last complete valid ownership snapshot; n/a means no partition ownership in that snapshot.
 C0-C5 identify consumers. Equal partition counts do not imply equal processing load.

Figure 23: Observed high-traffic partition ownership in the five-minute redistribution trials. Baselines retained all twelve on Consumer 2; redistribution placed four on each existing consumer. Counts use the final complete valid ownership snapshot; absent consumers are marked n/a.

Acknowledged but unfinished work - 700 messages/s



Each trial: 1 min warm-up + 5 min evaluation + 2 min drain. Common scales.
 Distinct ACK/completion events, including warm-up; this is not broker offset lag.
 Dotted line: scheduled intervention. Shading: release requested to active verified.

Figure 24: Distinct acknowledged messages not yet completed during the five-minute redistribution trials, including warm-up work. Event records show unfinished work decreasing after the intervention despite gaps in offset-based lag measurements. These curves are not the evaluation-only unfinished-message percentage at the drain cutoff.

5.2 Persistent-hot and partition-lag diagnostics

These two earlier blocks each used a five-minute evaluation, one-minute warm-up and two-minute drain, with 210,000 successfully published evaluation messages per trial. Each block contains its own unchanged-assignment baseline and its intervention, each run twice. They remain separate comparisons rather than being pooled into the later ten-minute experiments. Tables 3, 4 and 6 collect the outcomes, resource use and lag summaries. The figures below add persistent-hot counts and maximum and mean partition lag.

The historical monitoring gaps are retained. Some predate the correction that initializes a returned-record position from a verified assignment offset while the client position remains unresolved. The correction was not applied retrospectively to invent missing values. Missing observations also restart the fifteen-observation persistence history, so the diagnostic gap may last longer than the ownership transfer itself. A shorter growth window cannot recover measurements that were never available. No comparison of these blocks with the later ten-minute blocks isolates duration alone: dates, selected partition identities and the monitoring implementation also changed. The figures use the same measurement definitions as the proposal. These runs have five minutes of evaluation followed by two minutes of drain; warm-up is excluded from the resource totals. The one-second SLO calculation includes overdue unfinished messages.

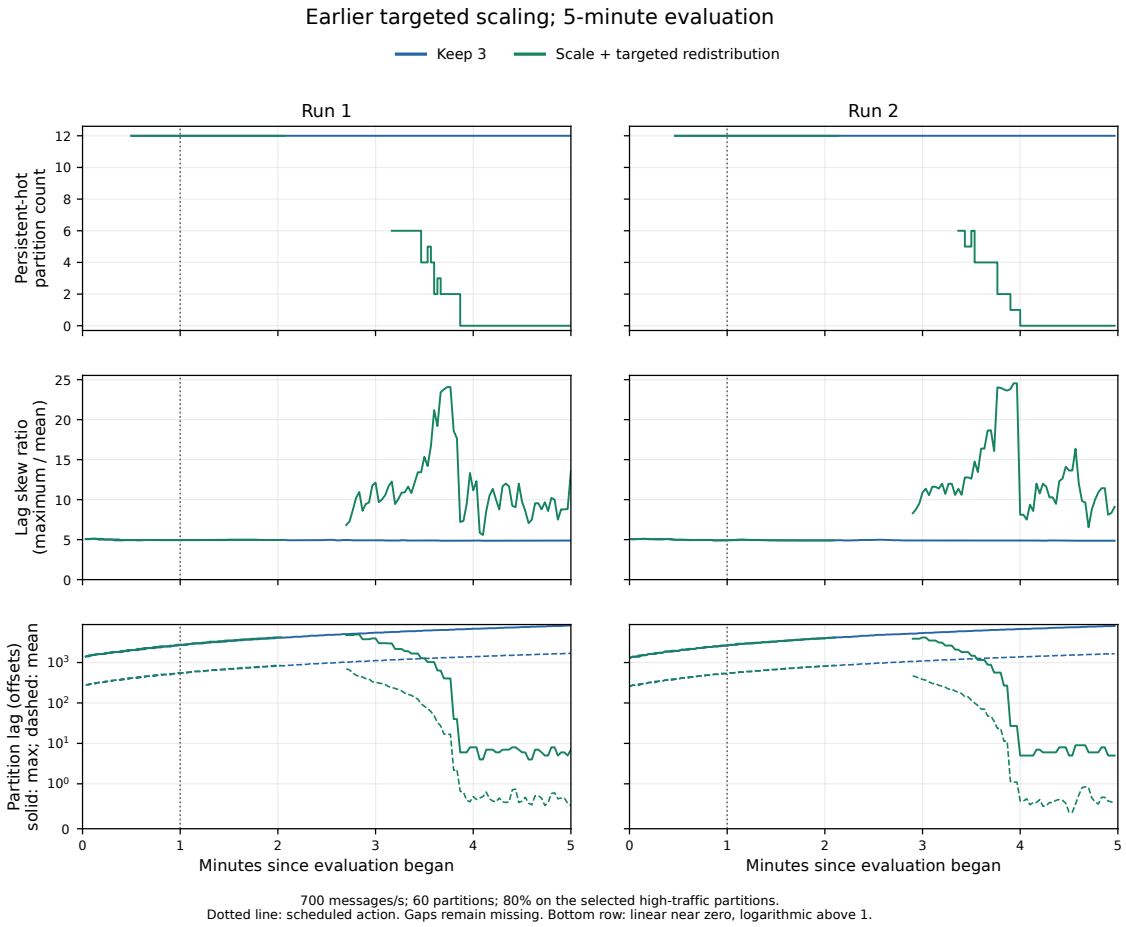


Figure 25: Partition-level measurements during five-minute scaling-with-reassignment trials. The panels show persistent-hot-partition count, lag skew, and maximum and mean partition lag. Missing observations are left as gaps.

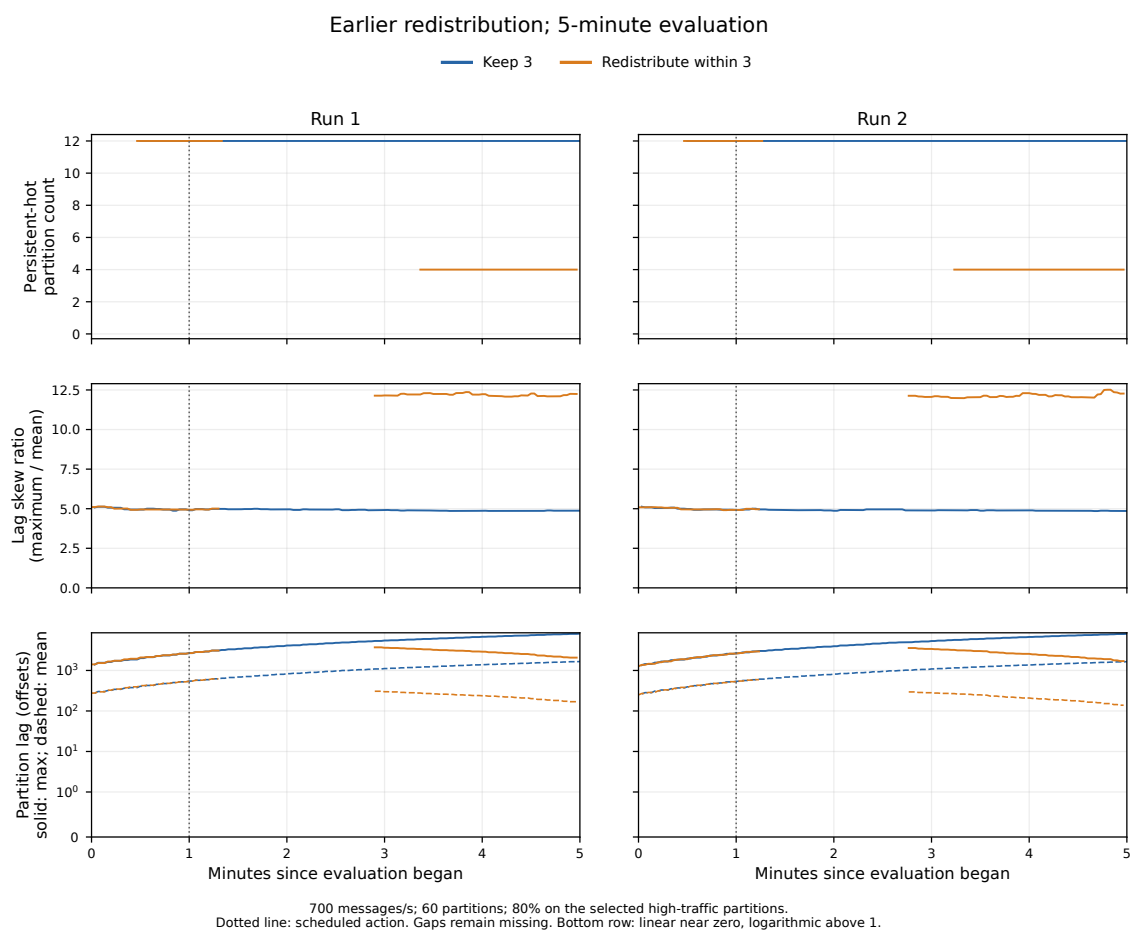


Figure 26: Partition-level measurements during five-minute reassignment trials with three consumers. The panels show persistent-hot-partition count, lag skew, and maximum and mean partition lag. Missing observations are left as gaps.

6 Completion-Deadline Sensitivity

The one-second SLO and half-second sensitivity threshold were applied retrospectively to these older trials. Overdue unfinished evaluation messages count as violations; no completion latency is assigned to them. The original 99-ms results remain in the experiment records.

Response	Run	Misses > 0.5 s (%)	Misses > 1 s (%)
Scaling with targeted redistribution			
Keep 3	1	83.447	83.418
Targeted scale 6	1	57.312	57.011
Keep 3	2	83.258	83.239
Targeted scale 6	2	58.572	58.171
Redistribution within three consumers			
Keep 3	1	83.438	83.413
Redistribute 3	1	61.794	61.591
Keep 3	2	83.278	83.241
Redistribute 3	2	61.100	60.840

Table 7: Completion-deadline sensitivity for the five-minute trials.

7 Detailed Plots: Ten-Minute Evaluation

These plots preserve the detailed views of the eight trials comparing unchanged ownership, redistribution among three consumers, scaling with targeted redistribution, and scaling with Kafka’s normal rebalancing. Each condition was run twice, at 700 aggregate messages/s with 80% of input directed to twelve of sixty partitions. Each trial used one minute of warm-up, ten minutes of evaluation and two minutes of drain. These are the same trials summarized in the main proposal, not additional experiments. Missing monitoring intervals remain visible; the figures have not been altered to fill gaps.

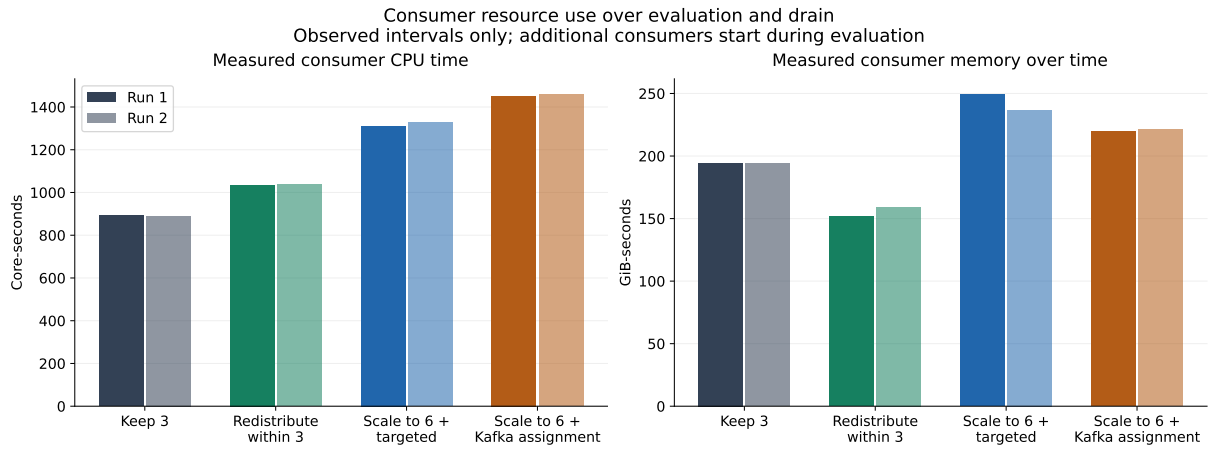


Figure 27: Measured consumer CPU time and integrated resident memory over evaluation plus drain for all eight trials. Bars sum covered process observations and exclude requested allocations, brokers and other node processes. Additional consumers begin during evaluation. Unchanged ownership leaves unfinished work, so its lower CPU total is not a like-for-like complete-processing cost.

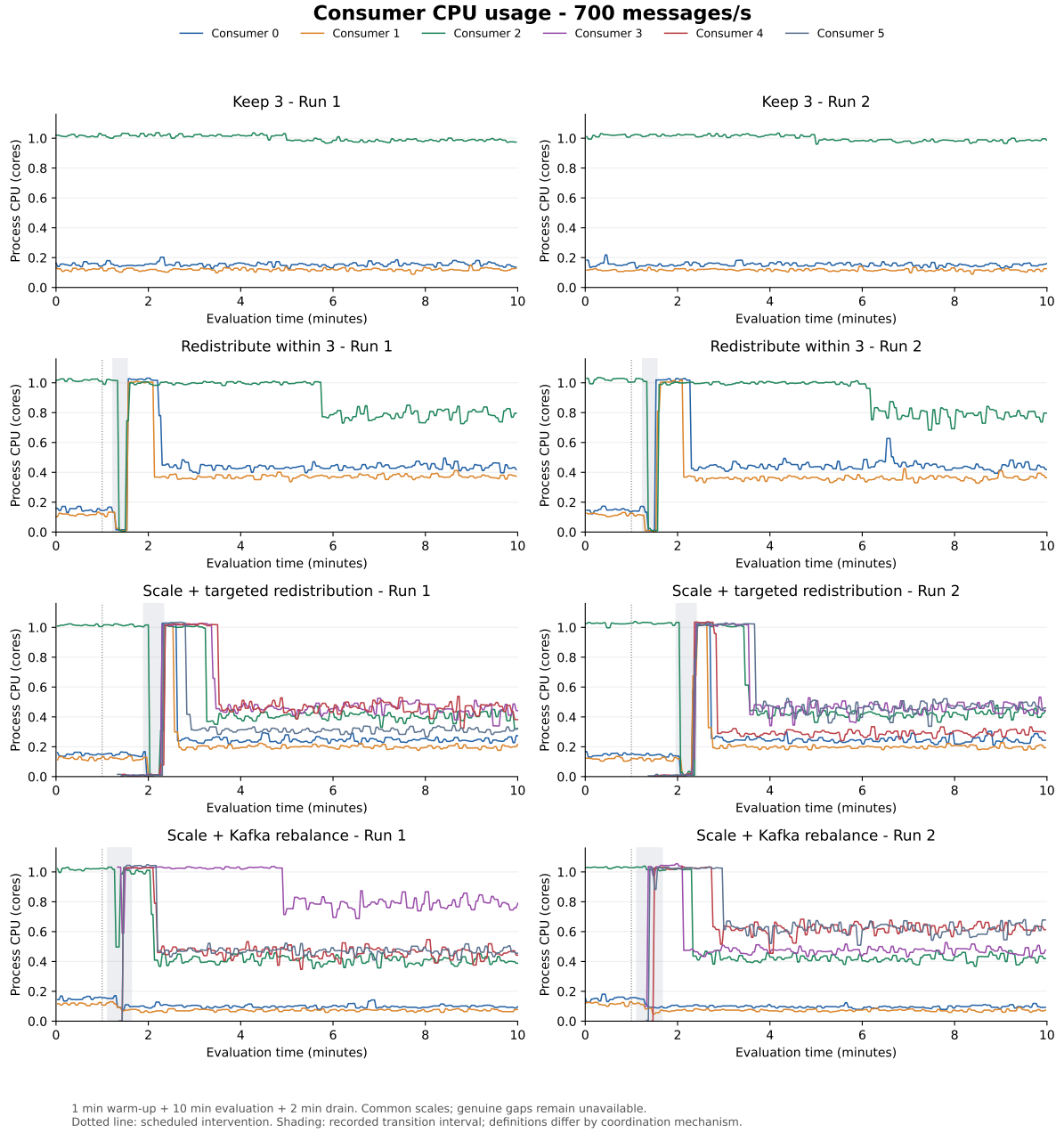


Figure 28: Observed consumer-process CPU use, in cores. These measurements describe process utilization during evaluation, not requested CPU, whole-node use or monetary cost. Each panel belongs to a separate thirteen-minute trial; the ten-minute evaluation is shown.

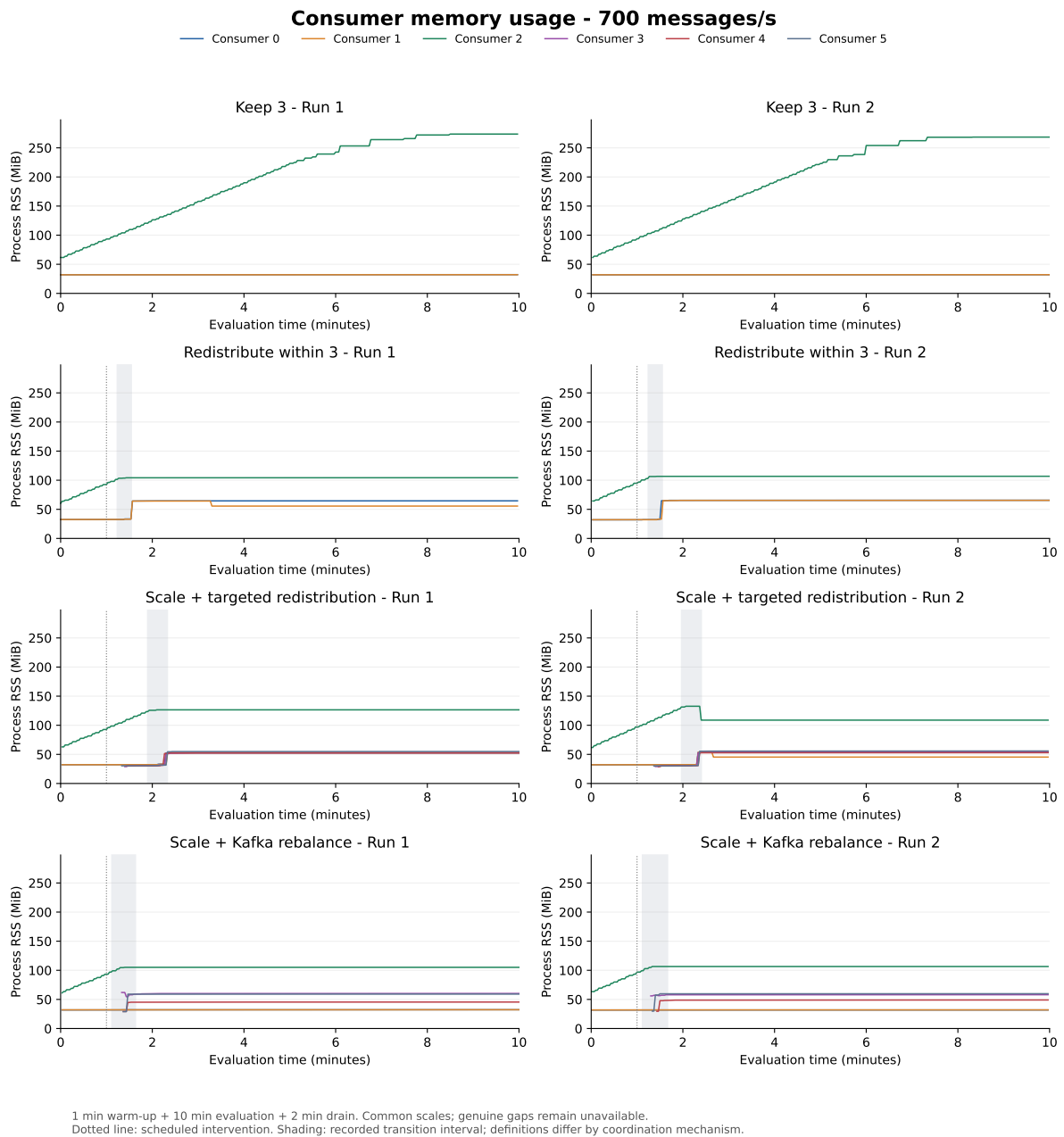


Figure 29: Observed consumer-process resident memory. RSS is distinct from requested Kubernetes memory and does not measure the volume of data moved during reassignment. Each panel belongs to a separate thirteen-minute trial; the ten-minute evaluation is shown.

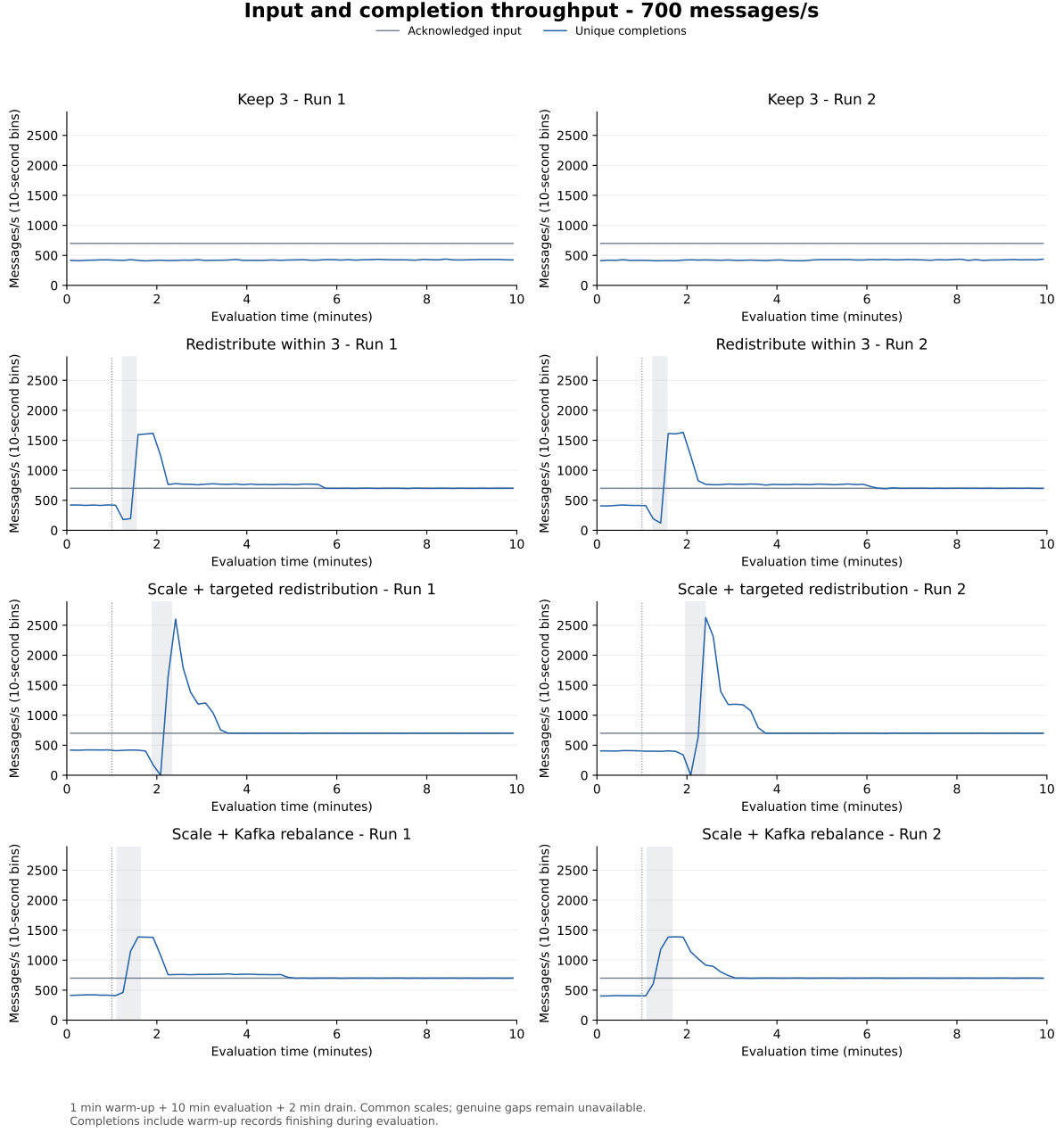
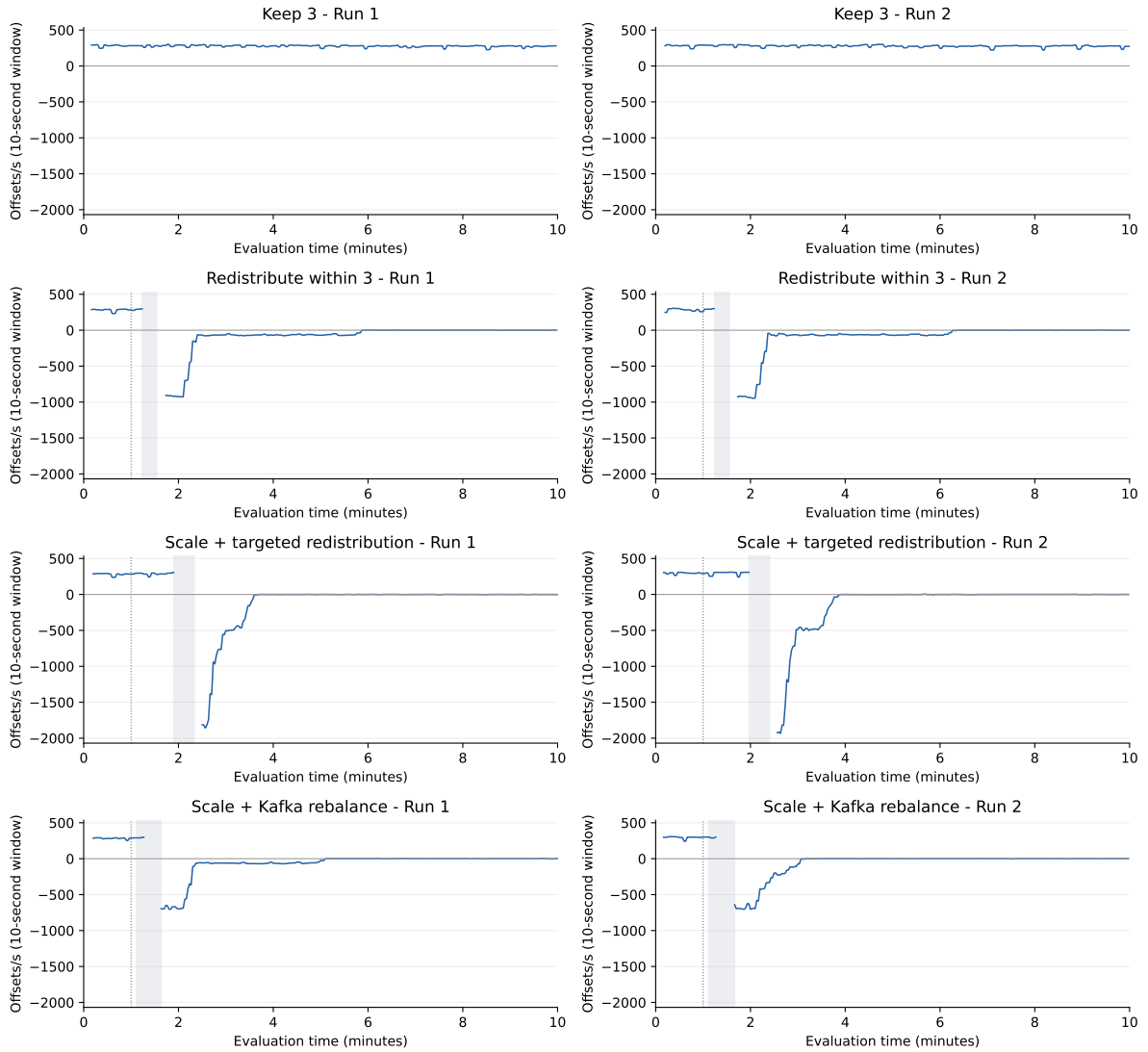


Figure 30: Acknowledged input and distinct completion throughput during evaluation. Completions include warm-up messages processed during this interval; latency and unfinished-message outcomes use the messages produced during evaluation through drain. Each panel belongs to a separate thirteen-minute trial; the ten-minute evaluation is shown.

Processing-backlog growth - 700 messages/s



1 min warm-up + 10 min evaluation + 2 min drain. Common scales; genuine gaps remain unavailable.
 Growth needs 10 seconds of fresh contiguous observations after a break. Dotted line: scheduled intervention.
 Shading: recorded transition interval; this differs from the growth-window waiting time.

Figure 31: Rolling processing-backlog growth over a ten-second window (five two-second intervals). This display window was shortened from thirty seconds after the trials and applied consistently to all eight trials. Positive values indicate accumulation and negative values indicate reduction. Missing observations, ownership changes and offset resets break the window; growth resumes only after a complete valid window. The separate requirement for thirty seconds of low backlog before confirming recovery is unchanged. Each panel belongs to a separate thirteen-minute trial; the ten-minute evaluation is shown.

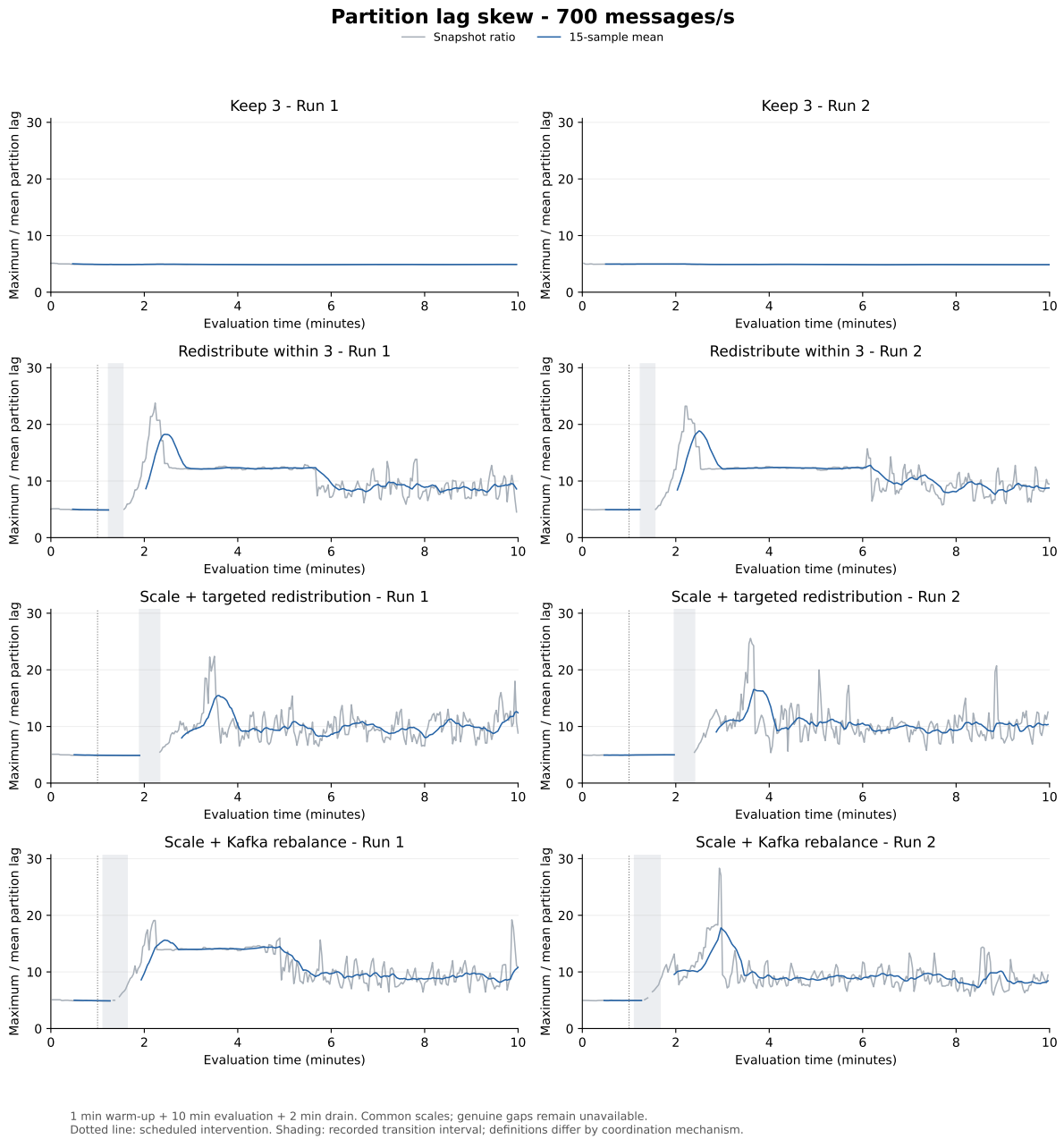


Figure 32: Partition-level lag skew during evaluation. Relative skew may rise when only a small absolute backlog remains; the ratio must be interpreted alongside total lag. Each panel belongs to a separate thirteen-minute trial; the ten-minute evaluation is shown.

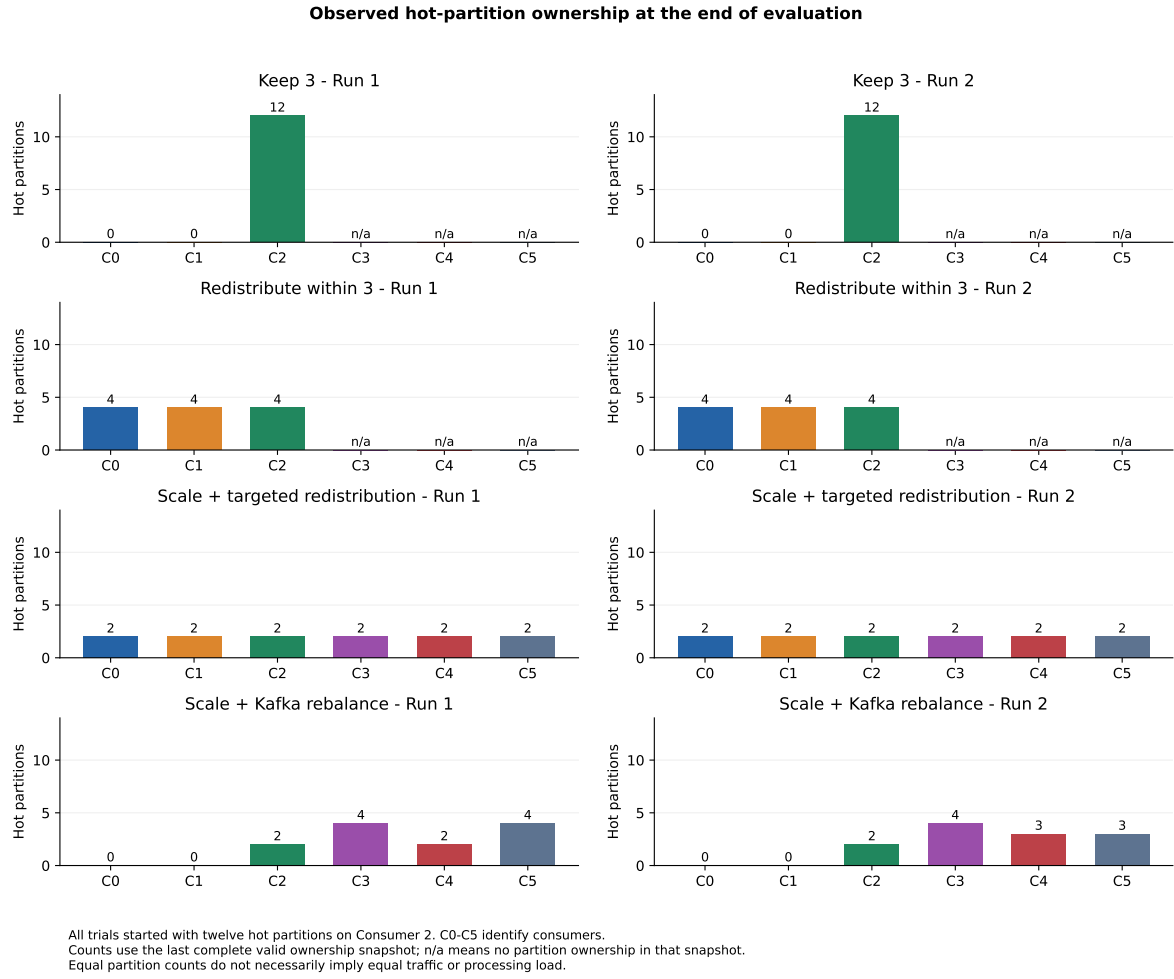
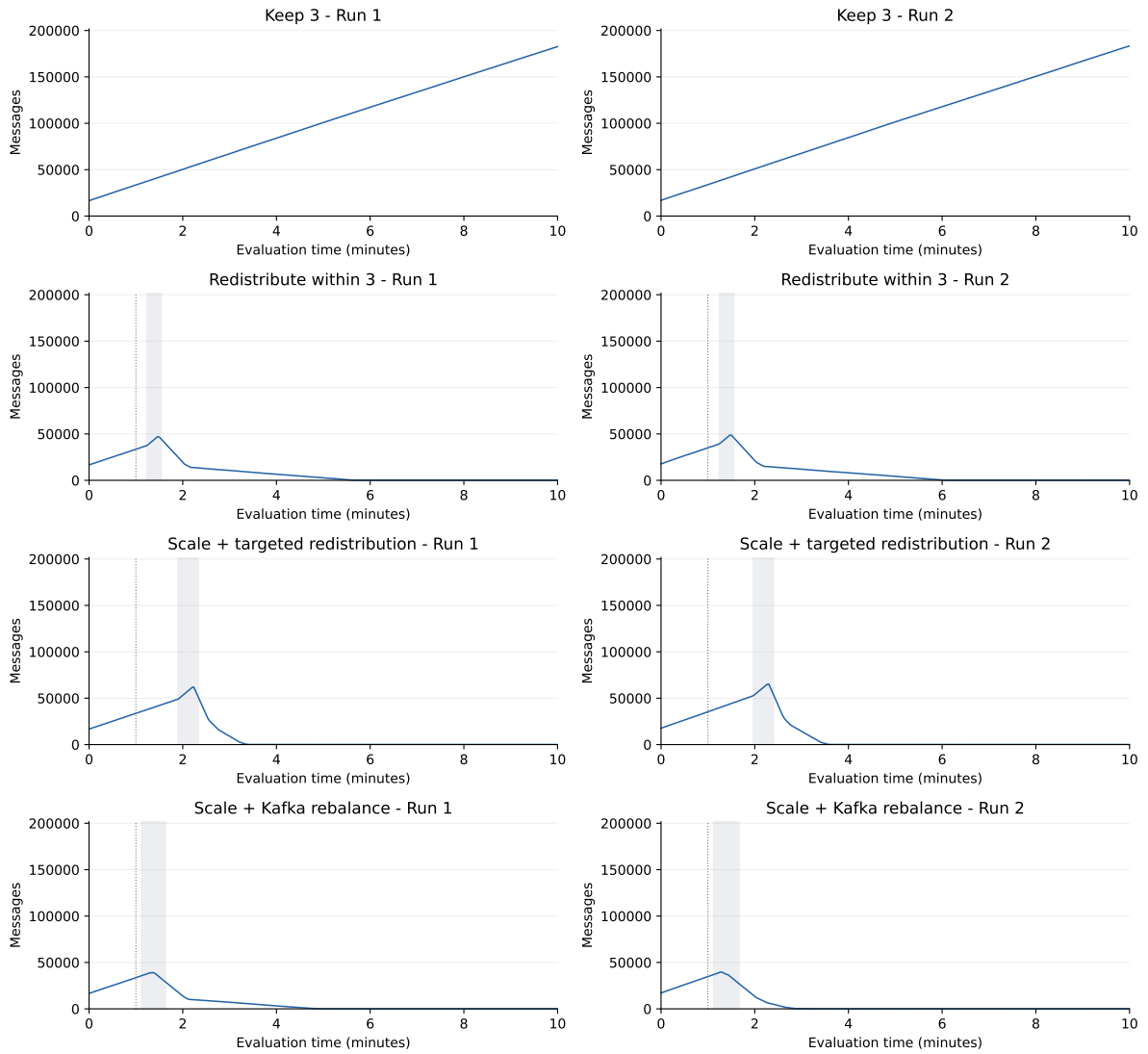


Figure 33: Observed partition ownership and high-traffic partition counts. Targeted and normal scaling need not reach the same final map. Equal partition counts do not establish equal assigned traffic or processing capacity. Each panel belongs to a separate thirteen-minute trial; the ten-minute evaluation is shown.

Acknowledged but unfinished work - 700 messages/s



1 min warm-up + 10 min evaluation + 2 min drain. Common scales; genuine gaps remain unavailable.
Distinct acknowledgment/completion events, including warm-up; this is not broker offset lag.

Figure 34: Distinct acknowledged messages not yet completed during the ten-minute evaluation of each thirteen-minute trial. Counts include warm-up work and use acknowledgment and completion events, rather than broker-offset lag. Intervention markers describe the recorded action and coordination intervals.

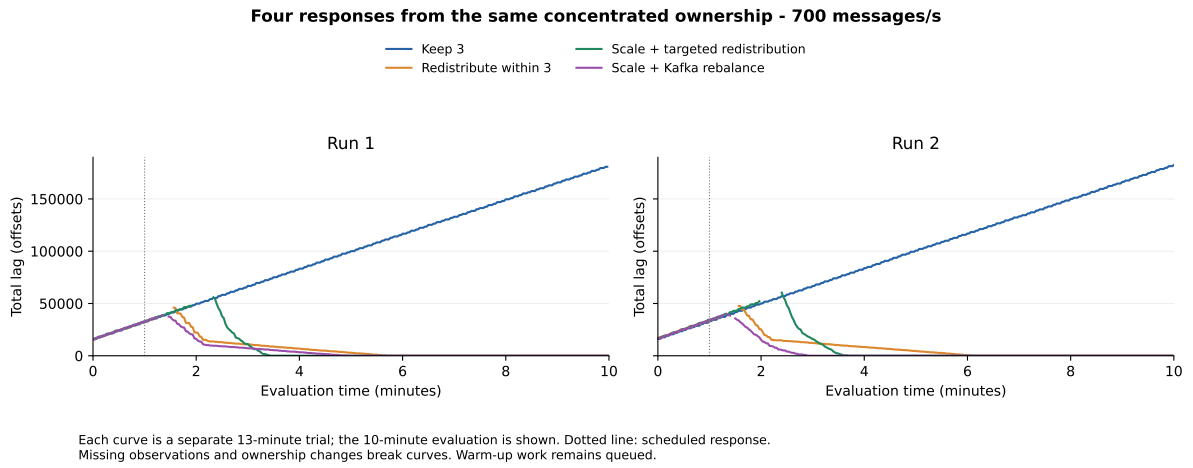


Figure 35: Total consumer-position lag during the ten-minute evaluation, comparing all four responses within each run number. Each curve represents a separate trial. Missing observations and ownership transitions remain gaps.

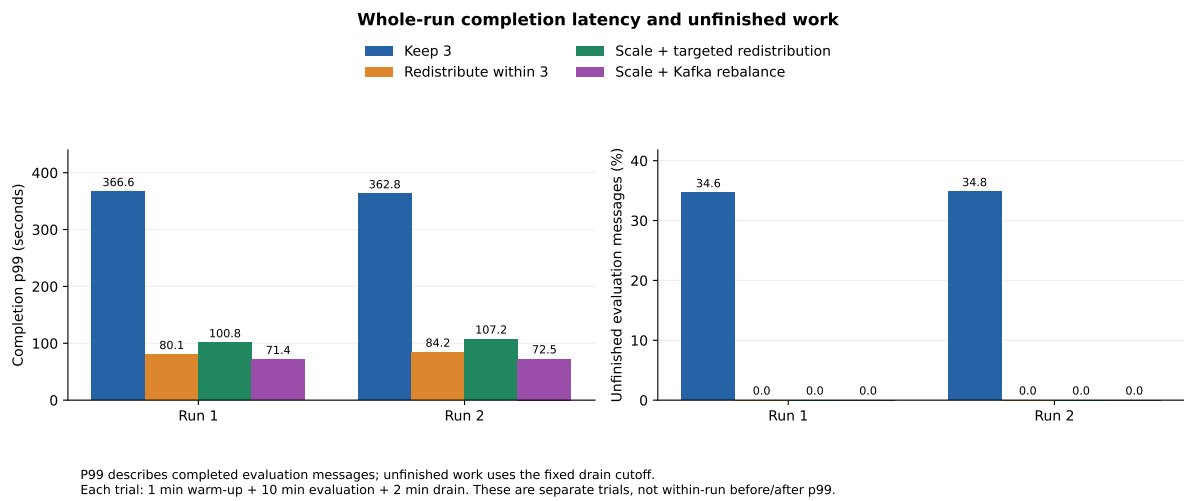


Figure 36: Whole-run completion p99 and unfinished evaluation messages at the fixed drain cutoff for the same eight trials. This preserves the earlier standalone presentation of outcomes also summarized in the main proposal.

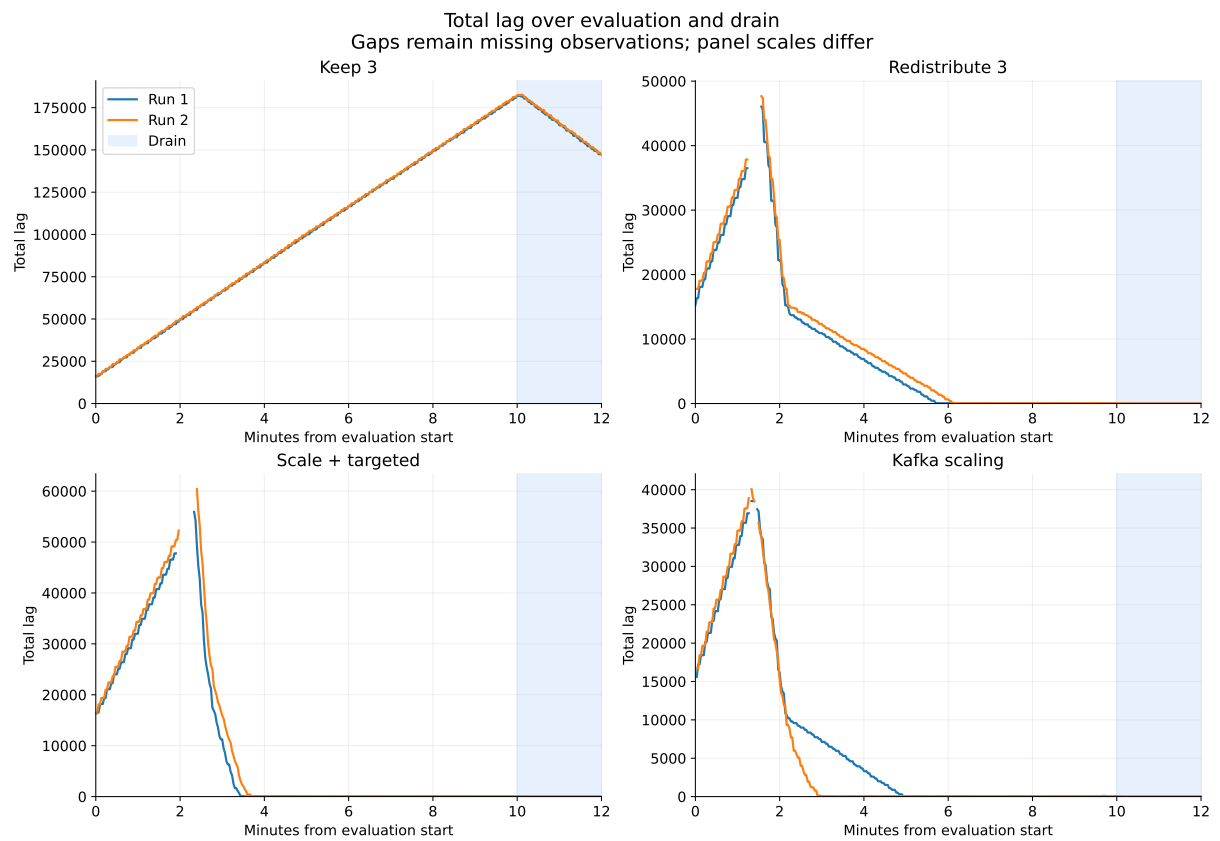


Figure 37: Total consumer-position lag through ten minutes of evaluation and two minutes of drain. The shaded interval is drain. Panel scales differ, and gaps remain missing observations.

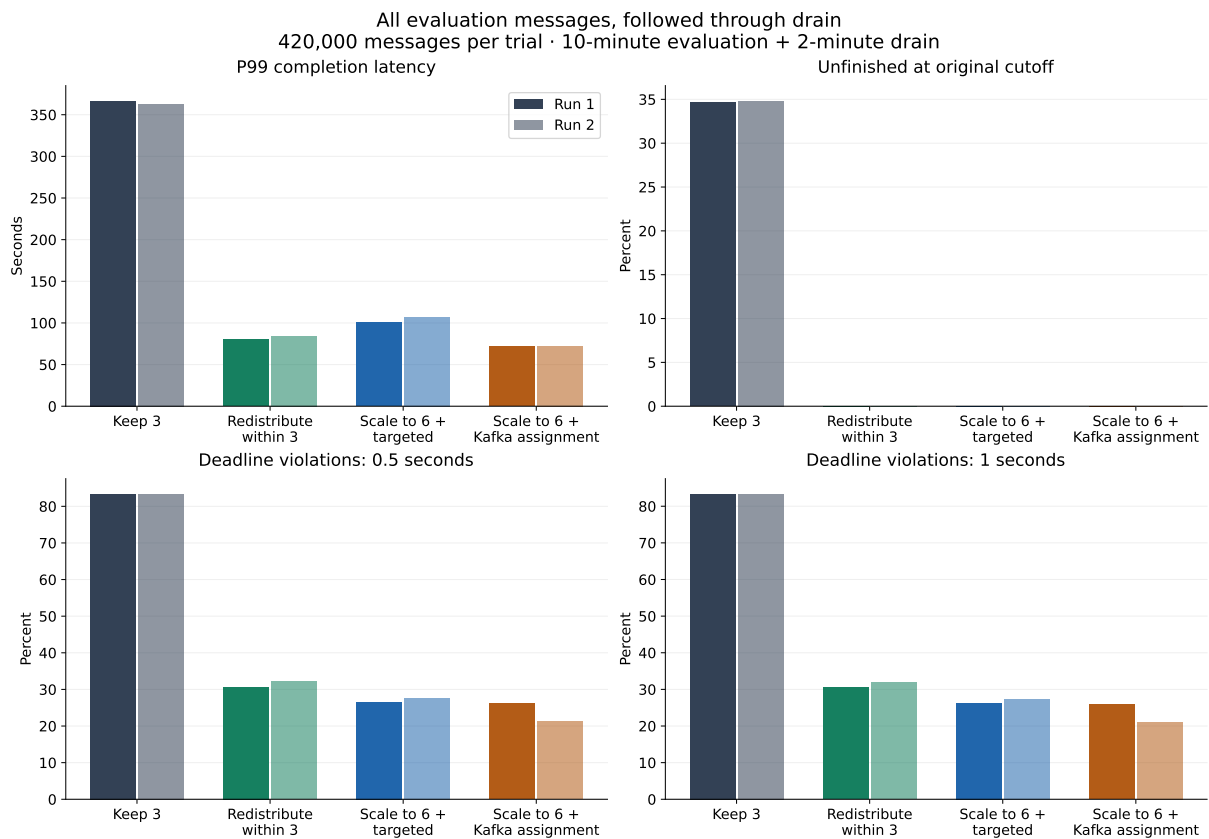


Figure 38: Archived combined outcome view for the same eight trials: completion p99, unfinished messages and deadline violations. The one-second deadline is the experimental SLO; the half-second result is a sensitivity check. Each trial has 420,000 acknowledged evaluation messages followed through drain.

8 Historical Scaling Campaign: Preserved Plots

These seven plots concern the earlier 24 performance trials: twelve comparisons of keeping three consumers versus scheduled scaling to six, with two runs per workload condition. Total target input was 1,500 messages/s, except for the lower-input controls at 600 messages/s. Each trial had one minute of warm-up and two minutes of drain. Evaluation lasted two minutes for the lower-input and short-pressure trials, nine minutes for sustained balanced pressure, and four minutes for the distributed-skew and single-partition trials. Total durations were therefore five, twelve and seven minutes, respectively. Historical panel titles retain their original production-duration labels, which include warm-up but exclude drain.

The original deployment, rates, outcomes and monitoring limitations are retained. These trials are not repetitions of the current 700-message/s study and do not establish its balanced operating rate. The full historical tables and discussion remain in `Old_Experiments.tex` in the Overleaf project. The figures and their original explanatory captions are preserved below.

p99 end-to-end latency

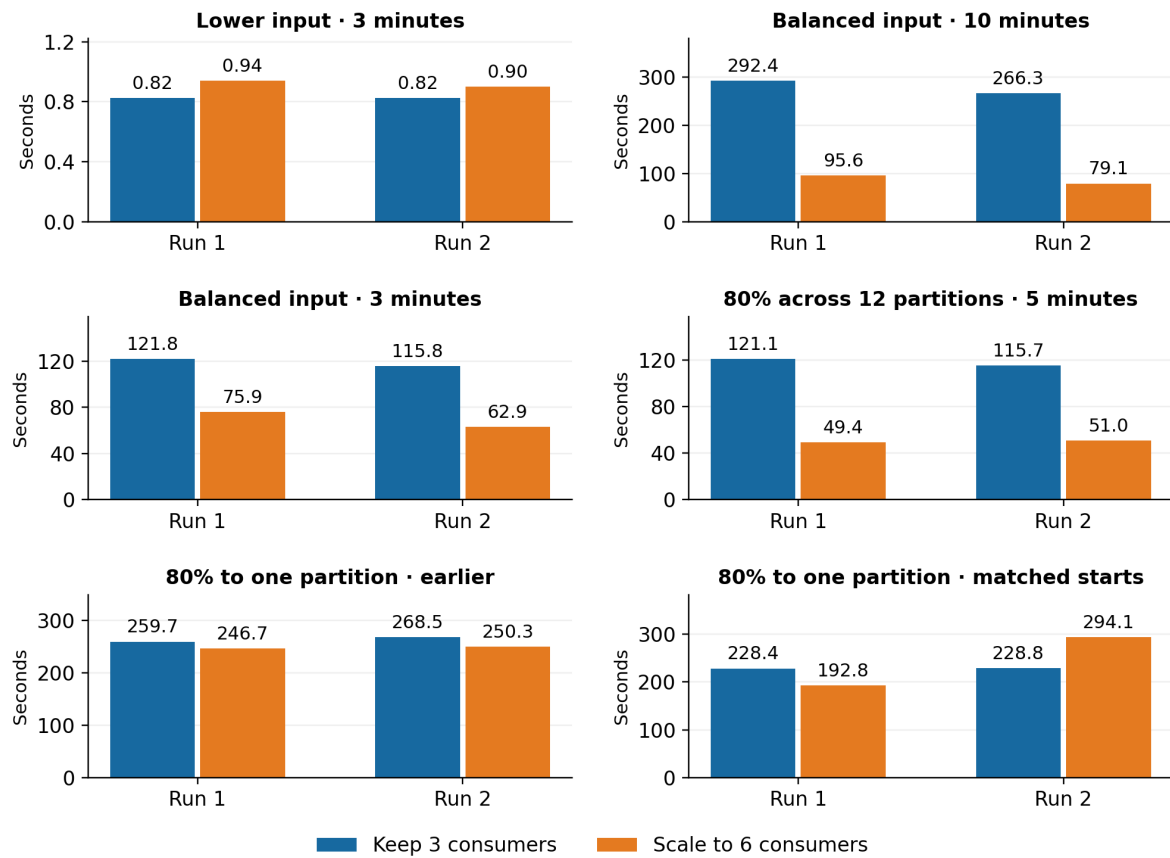


Figure 39: Run 1 and Run 2 each represent two separate trials: Keep 3 and Scale to 6. Each bar shows one trial; run numbers are not before/after observations. Latency is measured only for evaluation messages that completed by the end of the additional processing period. The lower-input panel uses a smaller vertical range so sub-second differences remain visible. No averages or logarithmic axes are used.

Unfinished messages

Unfinished messages at observation end

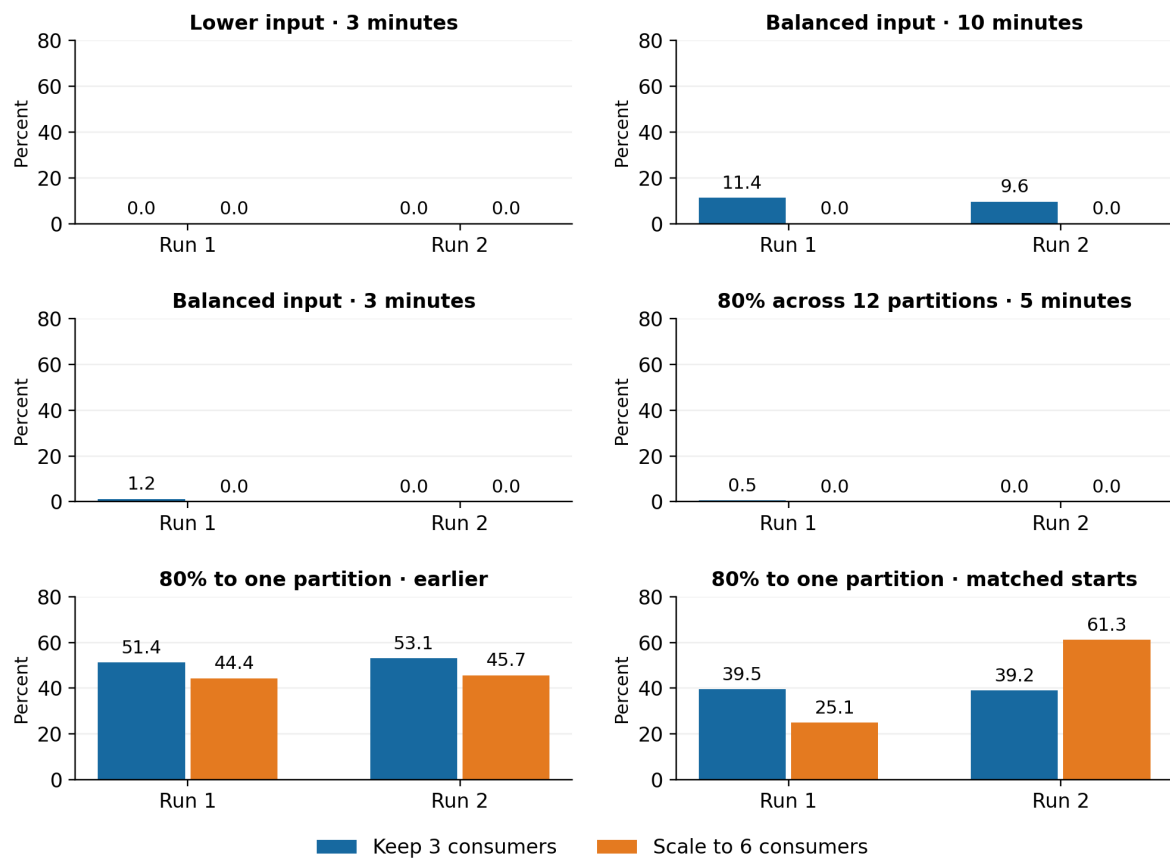


Figure 40: Bars show the percentage of acknowledged evaluation messages still unfinished after the additional 120-second processing period. All panels use the same vertical scale. A labelled zero means that all these messages finished.

Total lag

Total lag over time

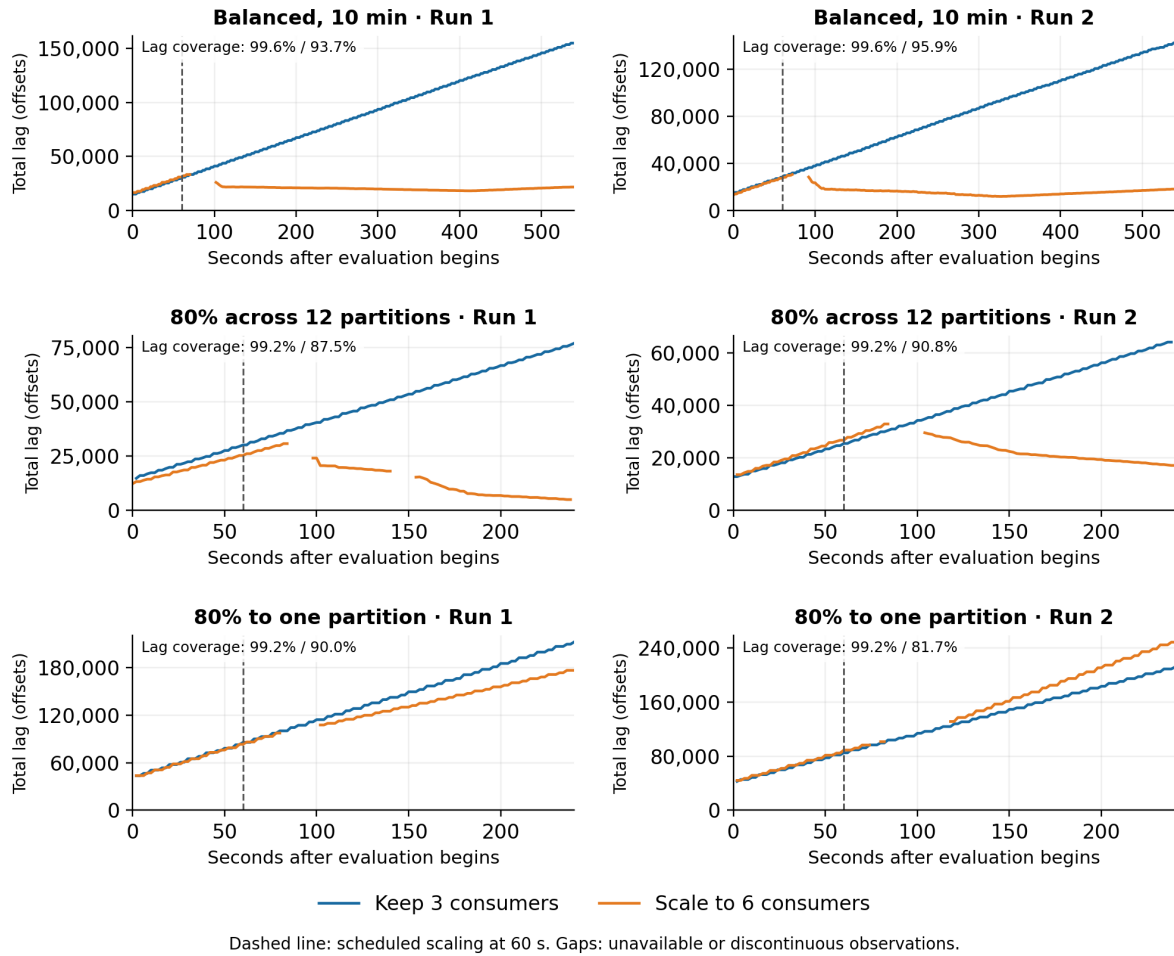
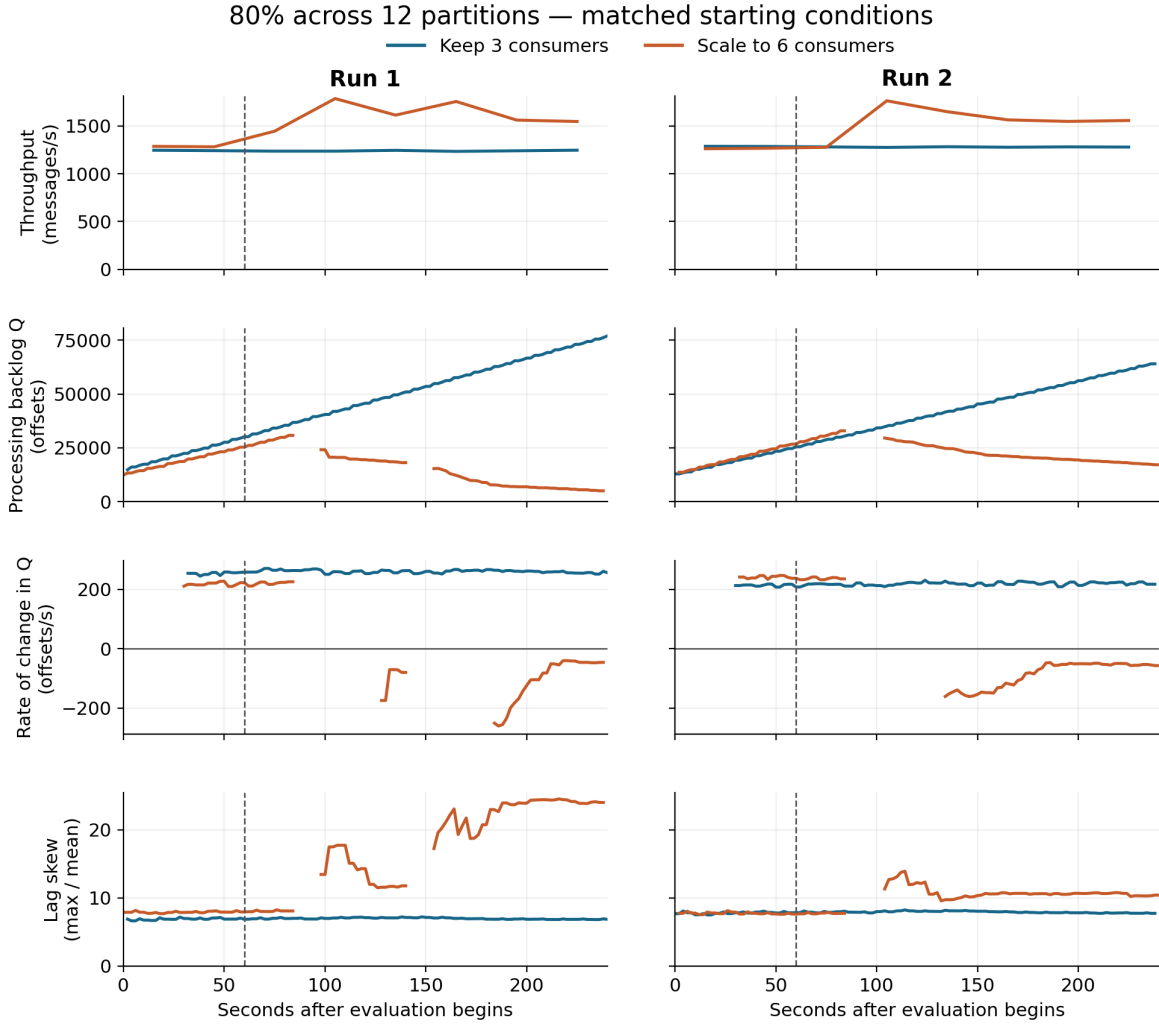


Figure 41: Total lag is the sum of partition lags based on the consumer position, consistent with the proposal definition. It is distinct from a count based on completed processing. Run 1 and Run 2 are shown for sustained balanced input and the later distributed-skew and single-partition workloads. Gaps and ownership changes are not connected. Coverage is reported in keep-three / scale-to-six order; comparisons below 90% in either run are diagnostic only and are not used for aggregate lag comparisons.

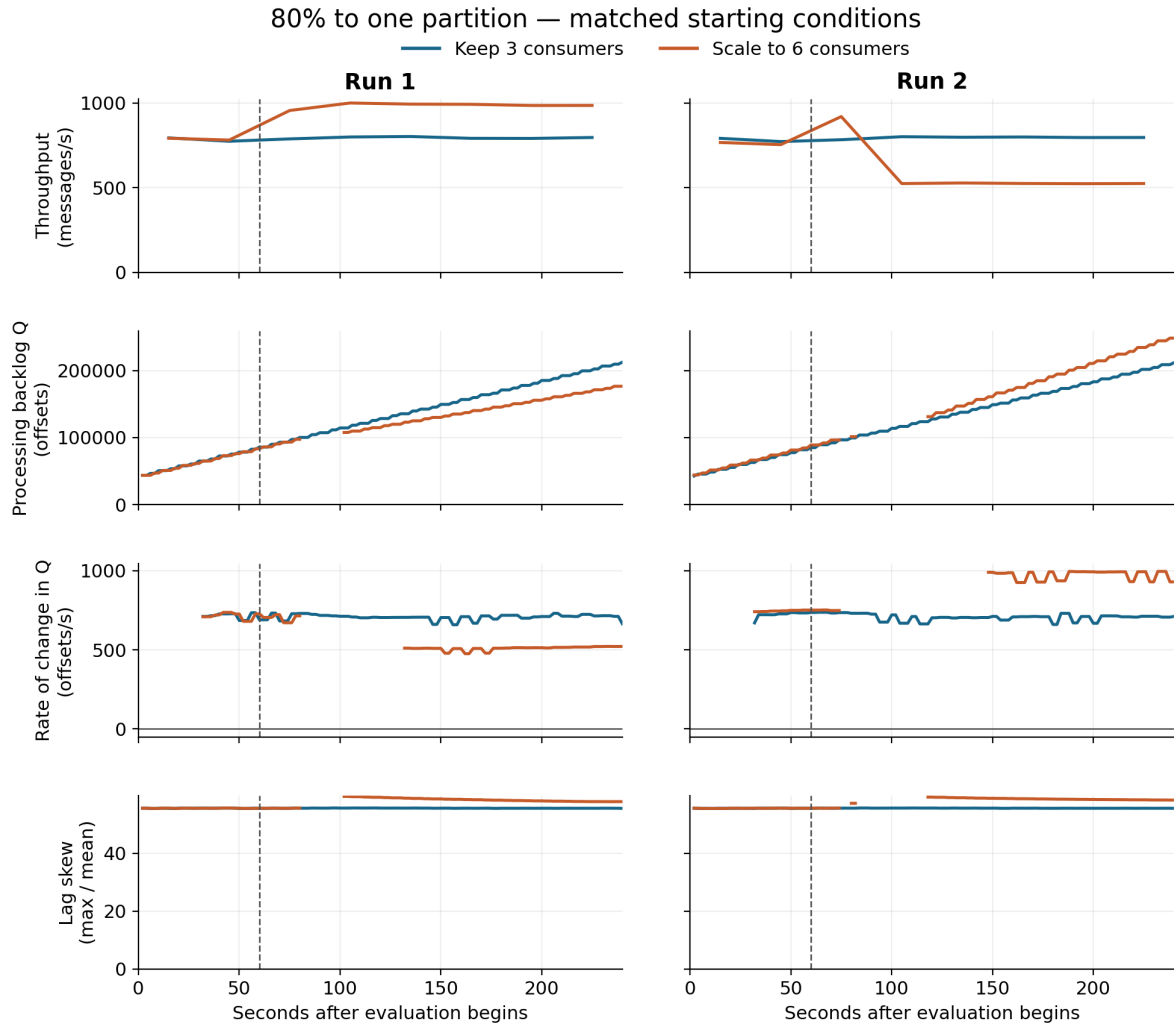
Performance under distributed skew



A flat positive growth line means Q keeps increasing; gaps are not zeros.
Dashed line: scheduled scaling at 60 s. Warm-up and drain are outside these plots.

Figure 42: Throughput counts distinct completions across all consumers in 30-second bins during evaluation, including any warm-up messages completed then. Processing backlog Q measures the offset distance from the contiguous processing-completion frontier to the log end; it is not the consumer-position lag B or the exact unfinished-message count at the drain cutoff. Growth is the change in Q over a contiguous window of approximately 30 seconds. A flat positive growth line means that backlog continues to increase. Lag skew remains $\max B_p / \text{mean} B_p$. Both scaling trials show higher throughput and declining Q in the available later intervals, while skew can remain high as total lag declines. Lag coverage (keep/scale) is 99.2/87.5% for Run 1 and 99.2/90.8% for Run 2. Gaps are unavailable observations; growth requires additional valid history. The lower-coverage comparison is diagnostic only for lag comparisons.

Performance under single-partition concentration



A flat positive growth line means Q keeps increasing; gaps are not zeros.
Dashed line: scheduled scaling at 60 s. Warm-up and drain are outside these plots.

Figure 43: The same definitions and plotting windows as the preceding figure apply. Both run comparisons are retained: scaling increases throughput in Run 1 but lowers it in the later intervals of Run 2. Processing backlog continues to grow in both scaling trials. The nearly horizontal positive growth curves therefore do not demonstrate stable backlog. Lag coverage (keep/scale) is 99.2/90.0% for Run 1 and 99.2/81.7% for Run 2; Run 2 is diagnostic only for lag comparisons. These curves show production-time behavior, whereas the outcome tables measure completion through the additional 120-second drain. Initial assignment and original consumer nodes were matched; placement after scaling and shared-machine variability were not controlled.

CPU and memory diagnostics: distributed skew

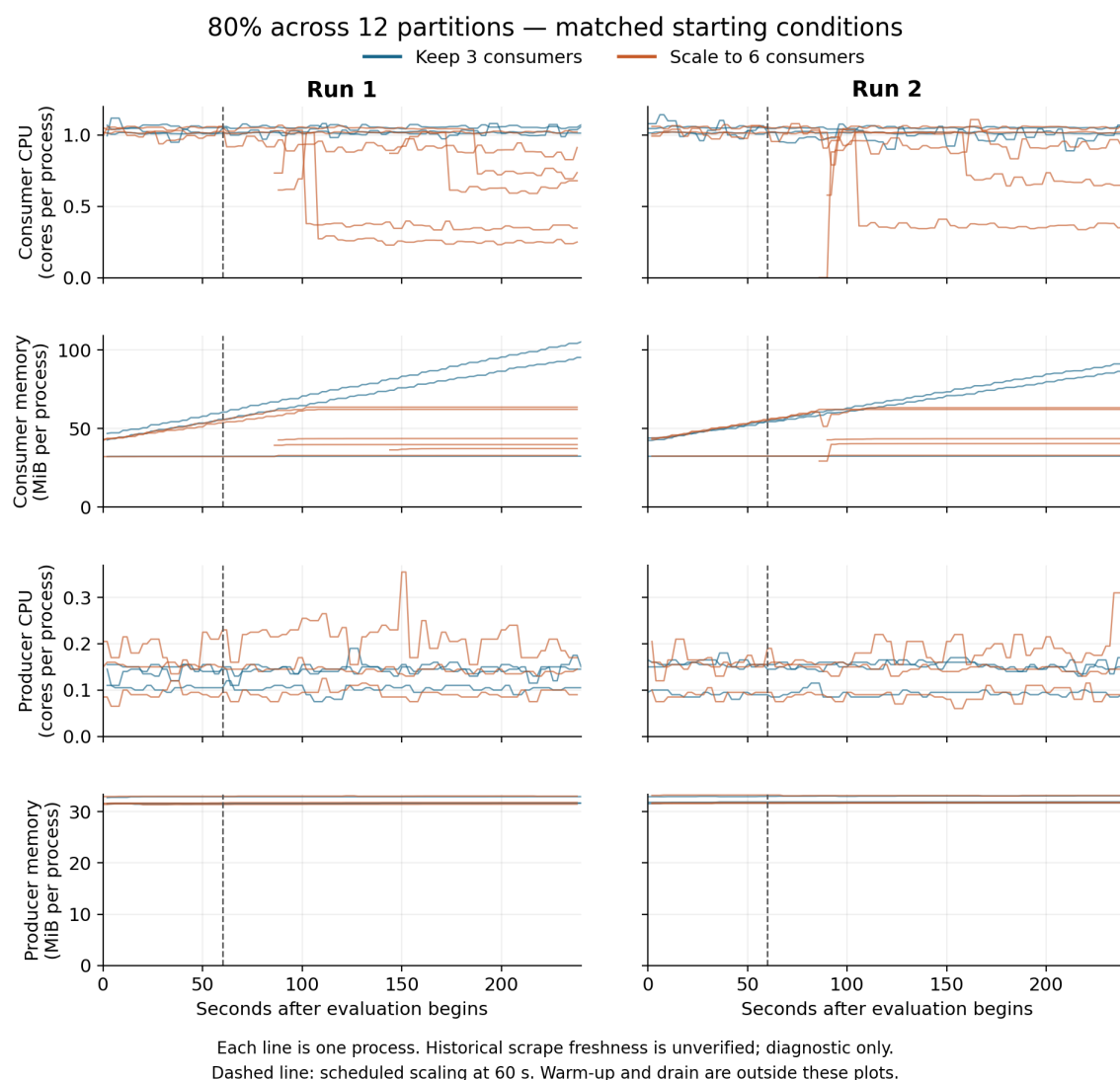


Figure 44: Each trace represents one producer or consumer process. CPU gauges are converted to cores, and resident memory (RSS) is shown in MiB. These are observed process measurements, distinct from requested CPU and memory resources over time. Historical resource-scrape freshness could not be verified, so repeated or flat gauge values cannot establish stable resource use. Missing intervals longer than three seconds are not connected. These plots are diagnostic supplements and do not establish total container usage, resource savings, memory saturation, or the cause of a performance difference.

CPU and memory diagnostics: single-partition concentration

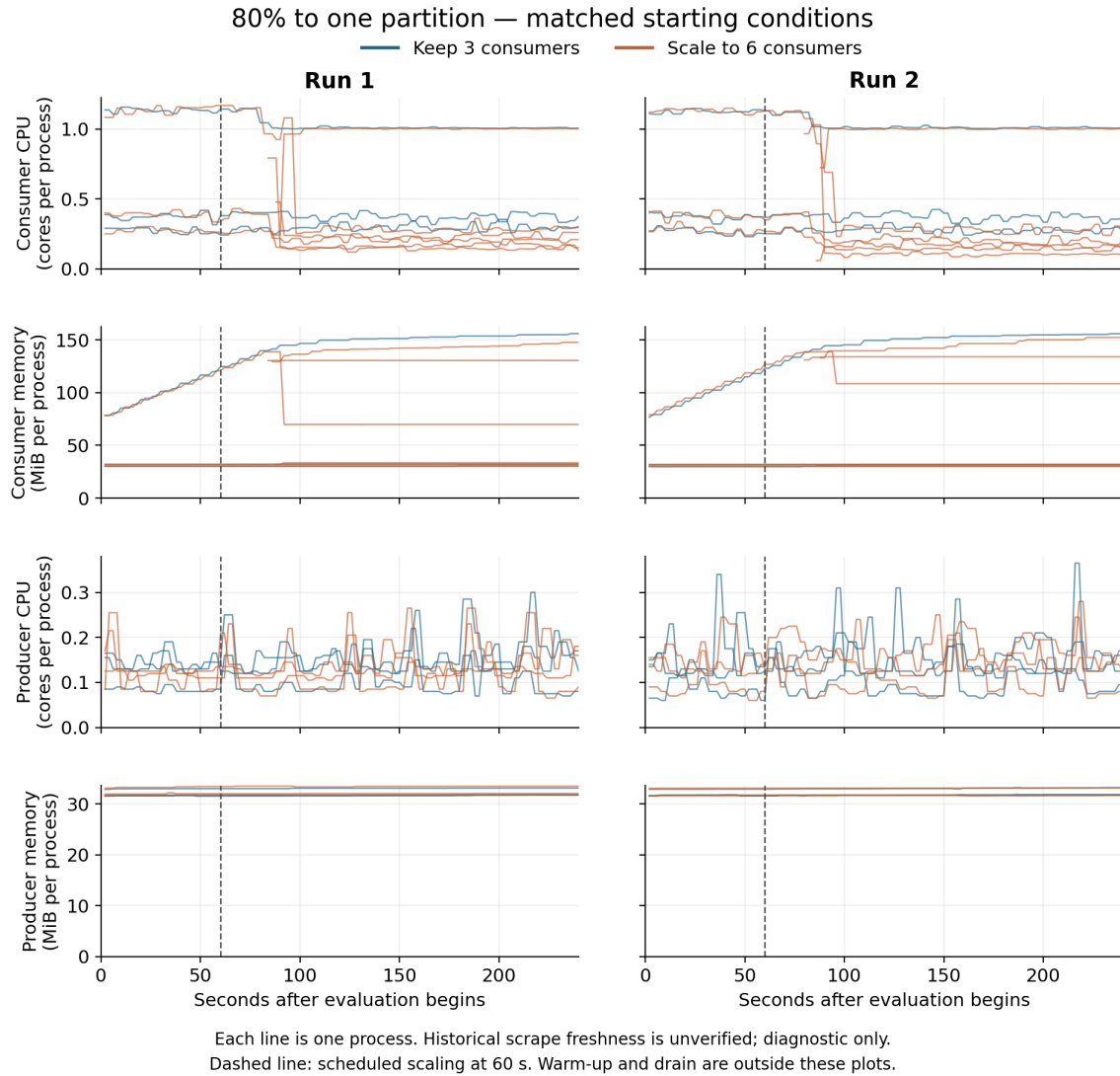


Figure 45: Both matched run comparisons are shown using the same process-level definitions and limitations as the preceding figure. A CPU trace near one core indicates the recorded usage of that process, not the capacity of the entire consumer group. Memory is resident process memory, not the Kubernetes memory request or total node memory. Original scrape freshness is unavailable. Consequently, these measurements cannot by themselves explain why scaling helped Run 1 and worsened Run 2. These historical figures describe only the original 24 scaling trials; the subsequent balanced-rate and fixed-ownership calibrations are reported separately in the main proposal and the five-minute sections of this supplement.

References

- [1] Amazon Web Services. *Monitor consumer lags*. Amazon MSK documentation. <https://docs.aws.amazon.com/msk/latest/developerguide/consumer-lag.html>. Accessed September 29, 2026.